

计算机组成与结构

(2026春季)

Pipelining

刘 铎

办公室: 信息大楼B822

邮箱: liuduo@cqu.edu.cn

我们能做得更好吗？

- What limitations do you see with the multi-cycle design?
- Limited concurrency
 - Some hardware resources are **idle** during different phases of instruction processing cycle
 - “Fetch” logic is idle when an instruction is being “decoded” or “executed”
 - Most of the datapath is idle when a memory access is happening

我们能利用空闲硬件来提高并发性吗？

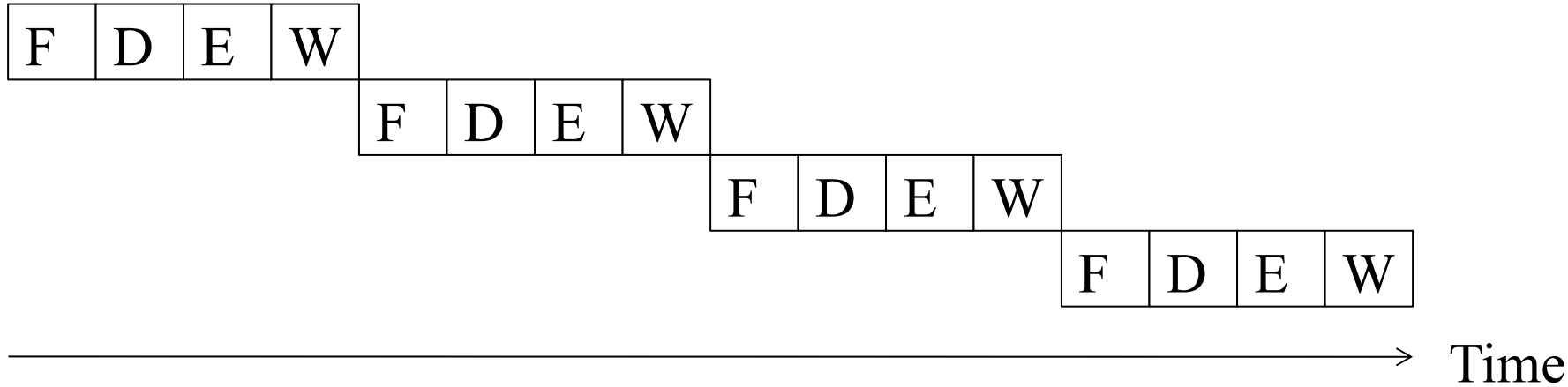
- Goal: **Concurrency** → **throughput** (more “work” completed in one cycle)
- Idea: When an instruction is using some resources in its processing phase, **process other instructions on idle resources** not needed by that instruction
 - E.g., when an instruction is being decoded, fetch the next instruction
 - E.g., when an instruction is being executed, decode another instruction
 - E.g., when an instruction is accessing data memory (ld/st), execute the next instruction
 - E.g., when an instruction is writing its result into the register file, access data memory for the next instruction

Pipelining: 基本思想

- More systematically:
 - Pipeline the execution of multiple instructions
 - Analogy: “Assembly line processing” of instructions
- Idea:
 - Divide the instruction processing cycle into distinct “stages” of processing
 - Ensure there are enough hardware resources to process one instruction in each stage
 - Process a different instruction in each stage
 - Instructions consecutive in program order are processed in consecutive stages
- Benefit: Increases instruction processing throughput ($1/\text{CPI}$)
- Downside: Start thinking about this...

示例：四条独立 ADD 指令的执行

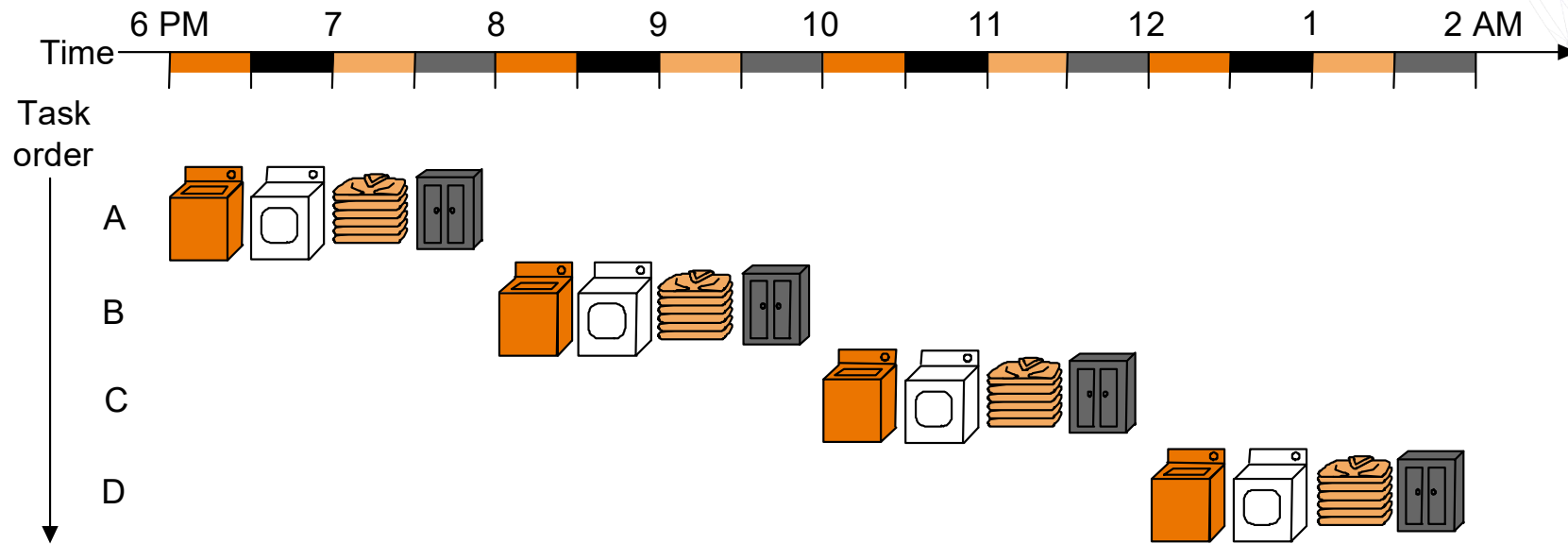
- Multi-cycle: 4 cycles per instruction



- Pipelined: 4 cycles per 4 instructions (steady state)



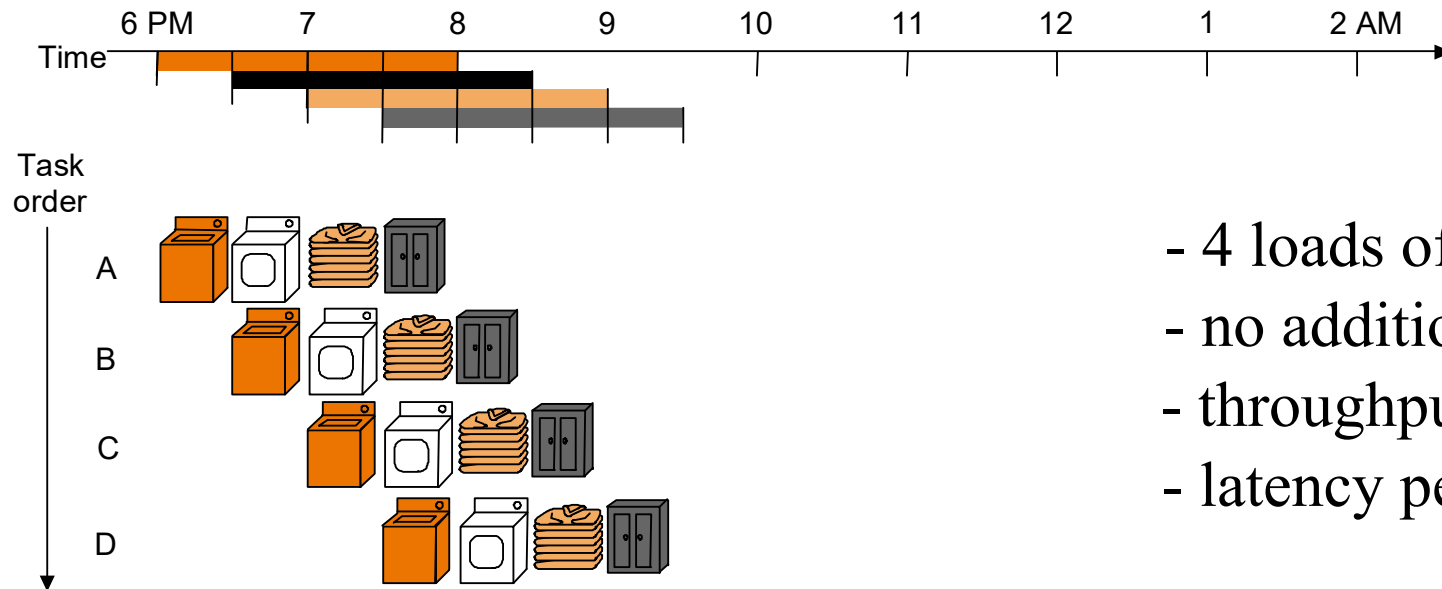
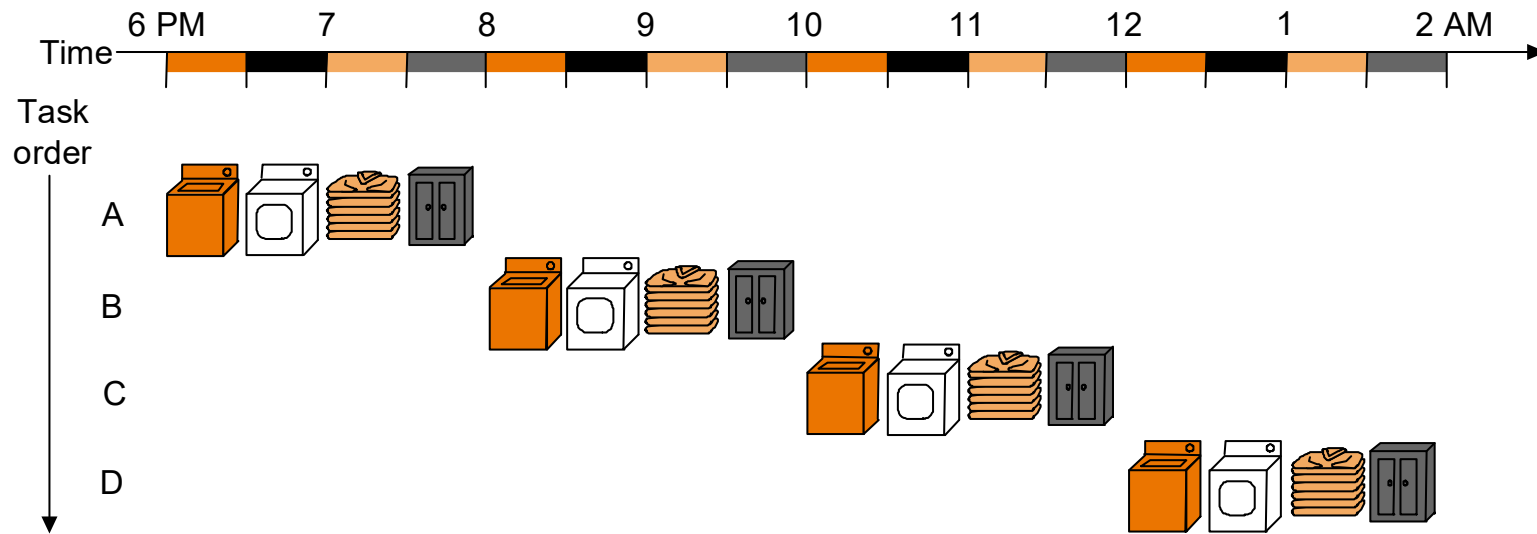
洗衣房类比 (The Laundry Analogy)



“place one dirty load of clothes in the washer”
“when the washer is finished, place the wet load in the dryer”
“when the dryer is finished, take out the dry load and fold”
“when folding is finished, ask your roommate (??) to put the clothes away”

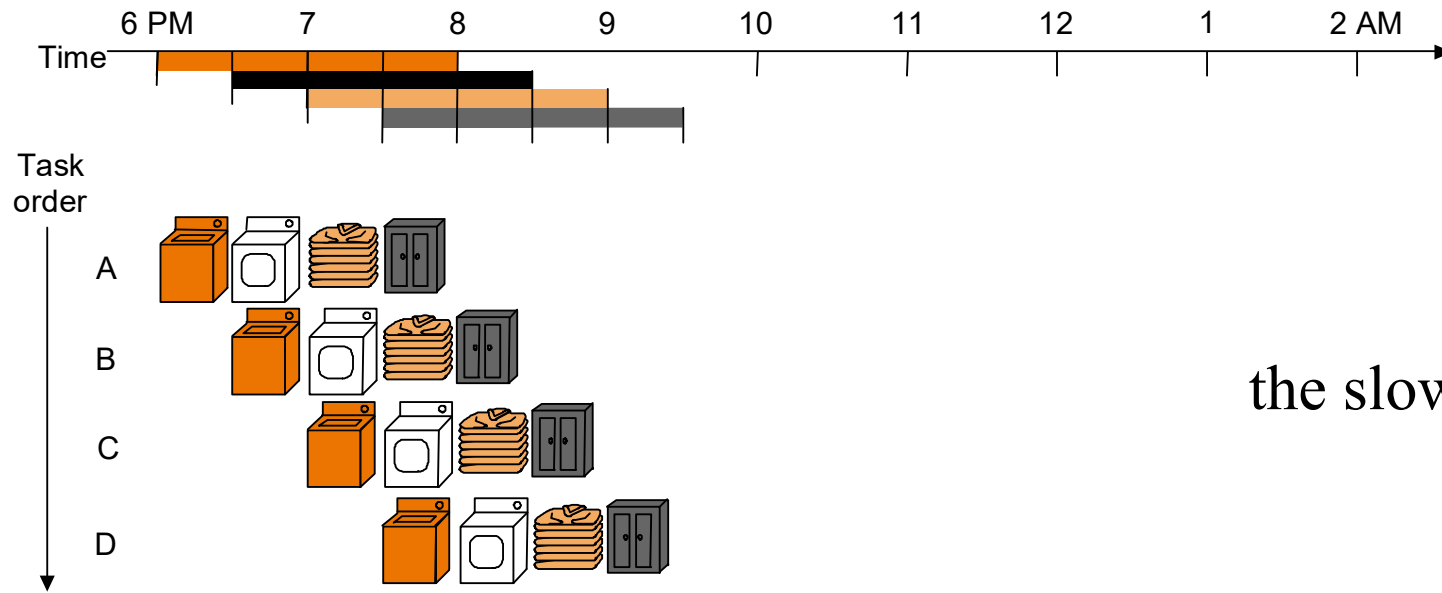
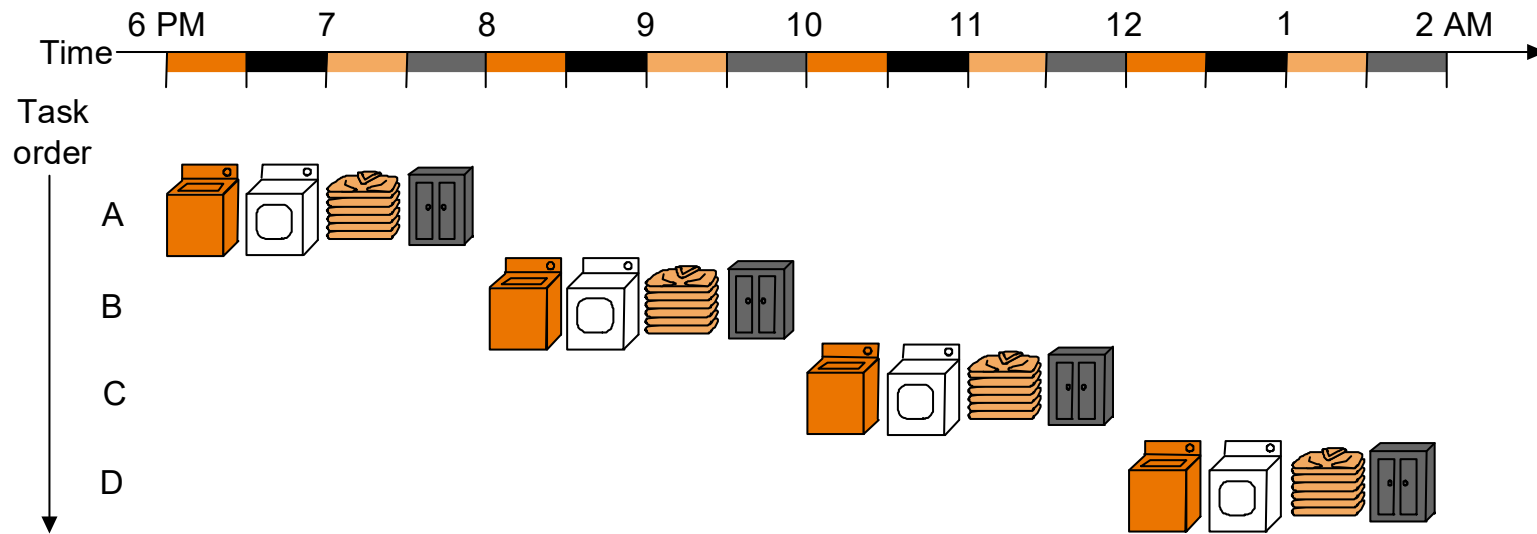
- steps to do a load are sequentially dependent
- no dependence between different loads
- different steps do not share resources

流水线处理多批衣物 (Pipelining Multiple Loads of Laundry)



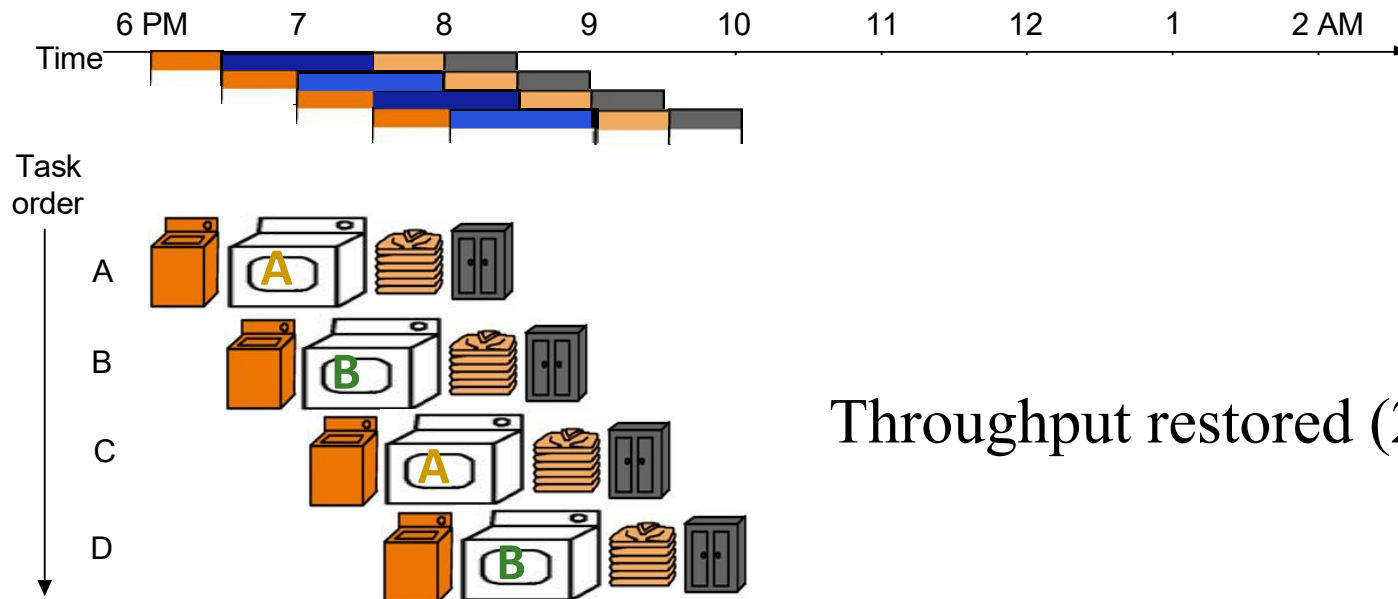
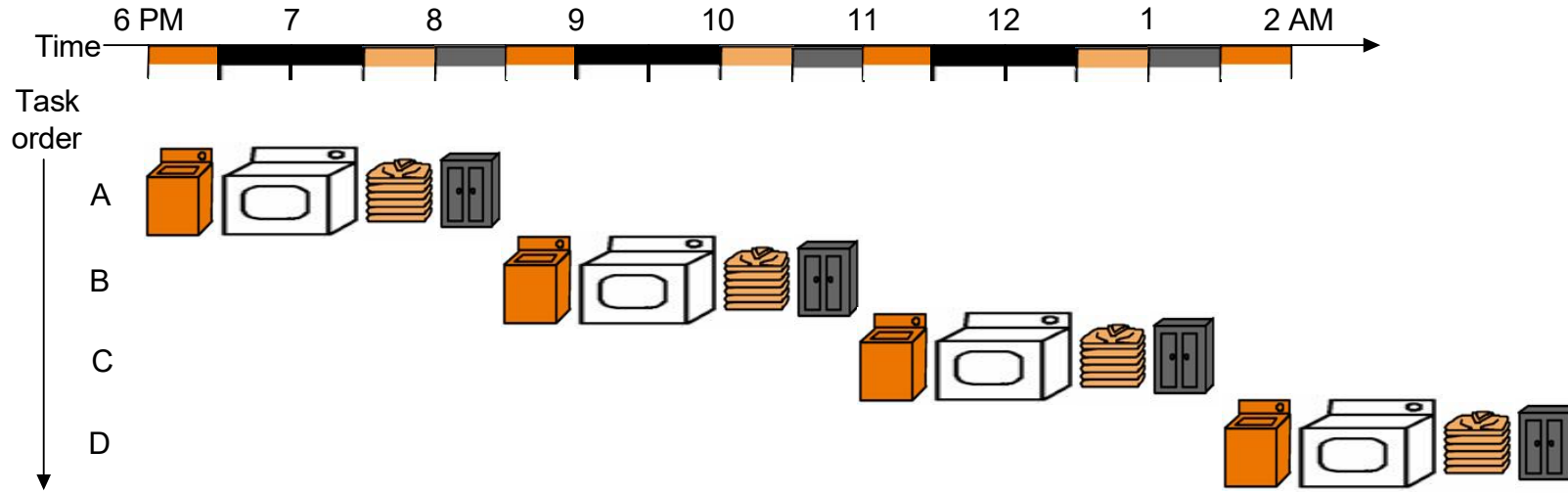
- 4 loads of laundry in parallel
- no additional resources
- throughput increased by 4
- latency per load is the same

流水线处理多批衣物：实际应用



the slowest step decides throughput

流水线处理多批衣物：实际应用

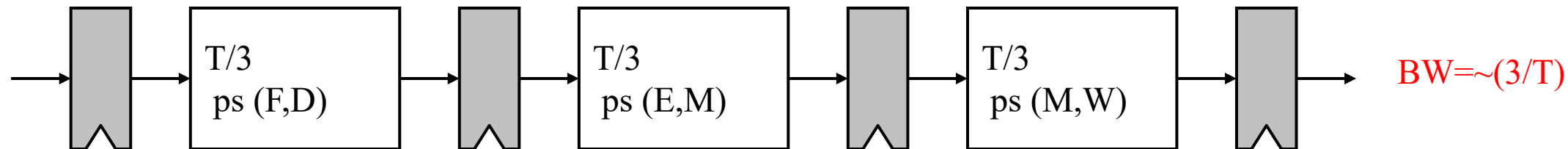
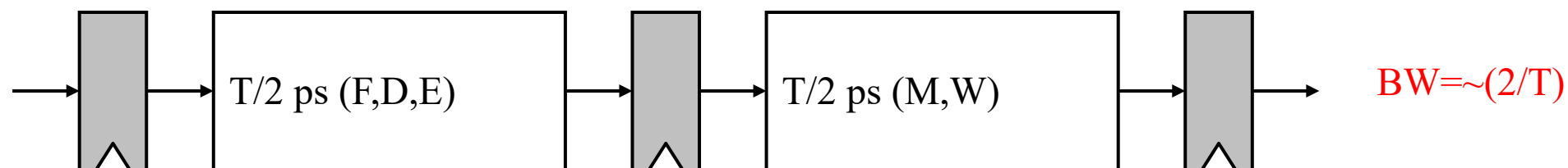
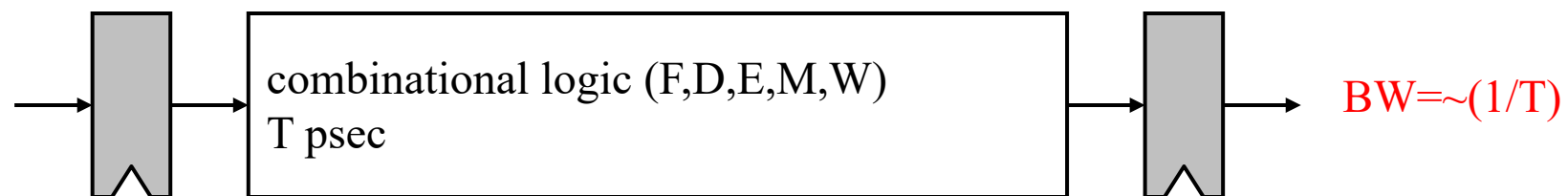


Throughput restored (2 loads per hour) using 2 dryers

理想的Pipeline

- Goal: Increase throughput with little increase in cost (hardware cost, in case of instruction processing)
- Repetition of identical operations
 - The same operation is repeated on a large number of different inputs
- Repetition of independent operations
 - No dependencies between repeated operations
- Uniformly partitionable suboperations
 - Processing can be evenly divided into uniform-latency suboperations (that do not share resources)
- Fitting examples: automobile assembly line, doing laundry
 - What about the instruction processing “cycle”?

理想的Pipeline



真实的流水线：吞吐率分析

- Nonpipelined version with delay T

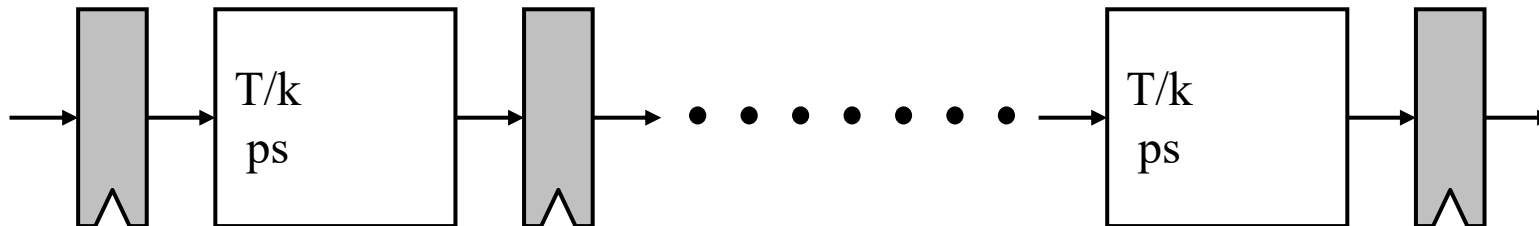
$$BW = 1/(T+S) \text{ where } S = \text{latch delay}$$



- k -stage pipelined version

$$BW_{k\text{-stage}} = 1 / (T/k + S)$$

$$BW_{\max} = 1 / (1 \text{ gate delay} + S)$$



真实的流水线：成本分析

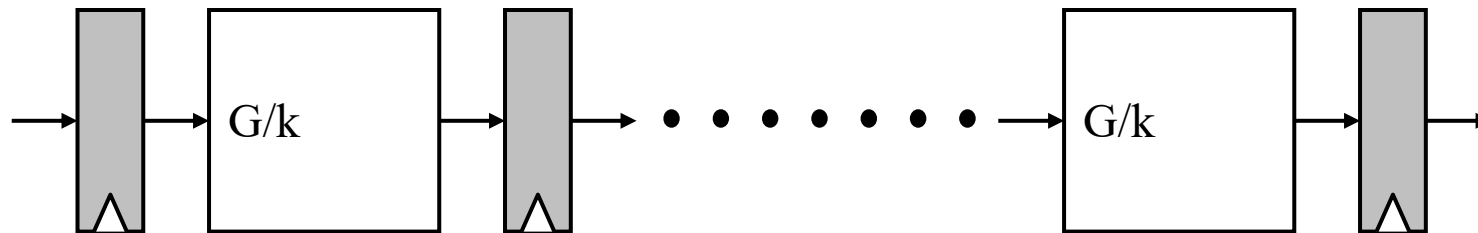
- Nonpipelined version with combinational cost G

$\text{Cost} = G + L$ where L = latch cost

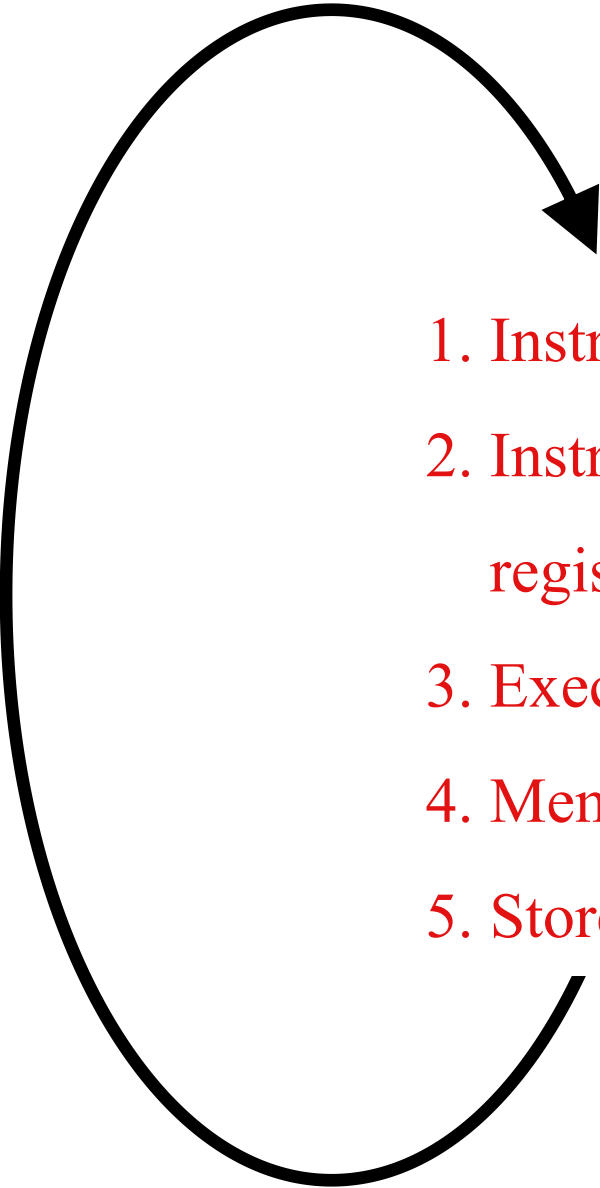


- k -stage pipelined version

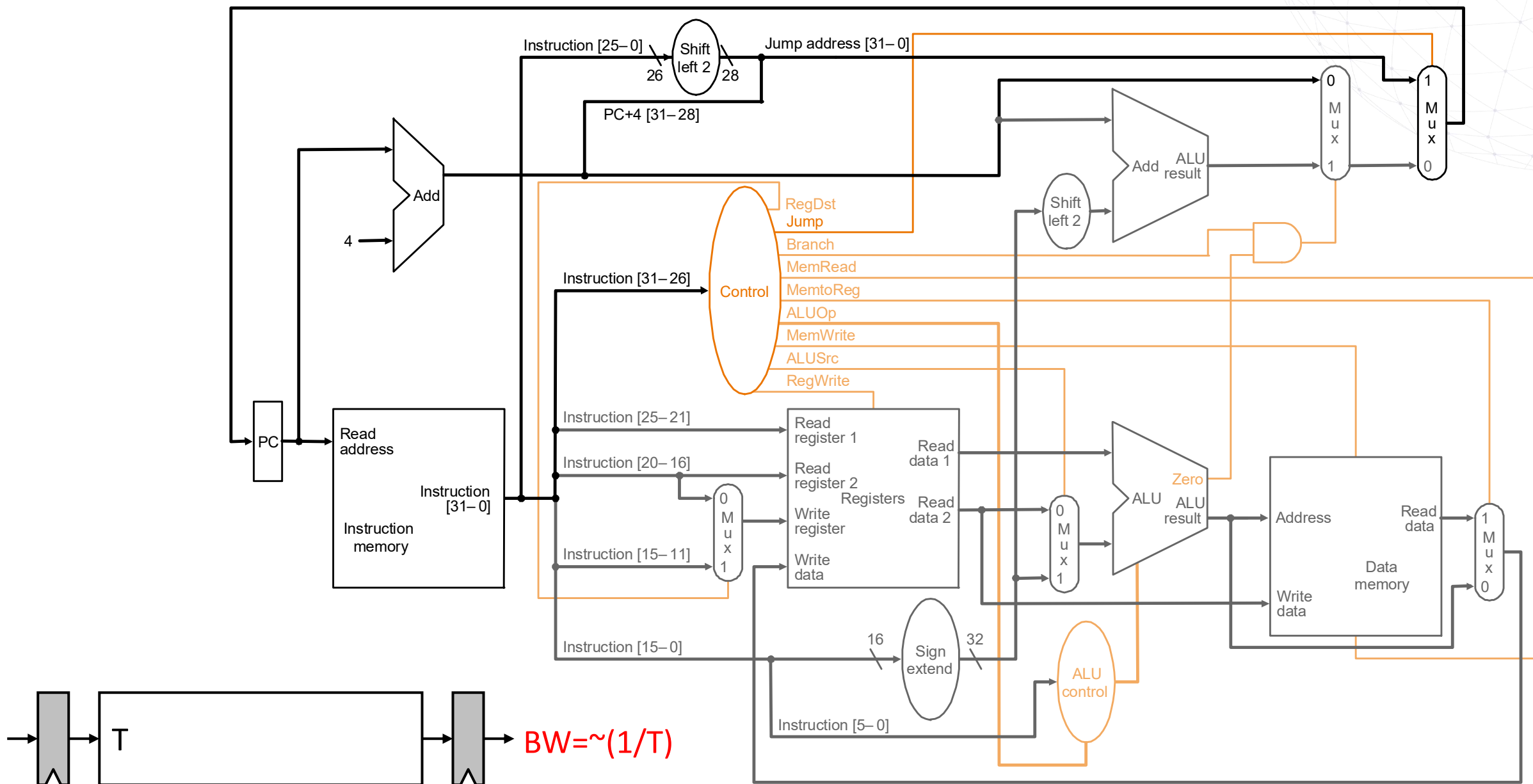
$\text{Cost}_{k\text{-stage}} = G + Lk$



指令处理周期

- 
1. Instruction fetch (IF)
 2. Instruction decode and
register operand fetch (ID/RF)
 3. Execute/Evaluate memory address (EX/AG)
 4. Memory operand fetch (MEM)
 5. Store/writeback result (WB)

单周期处理器设计



划分的阶段

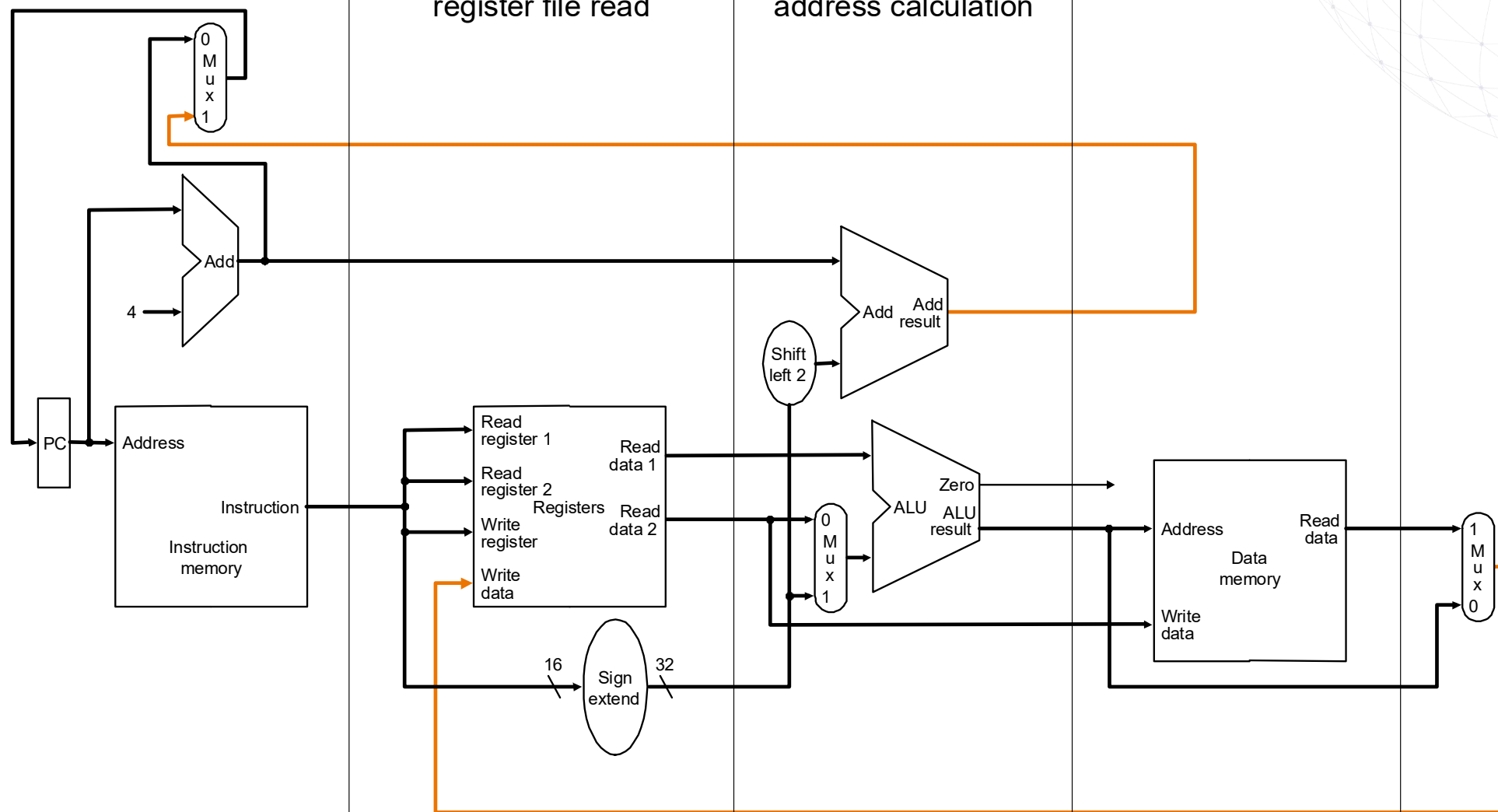
IF: Instruction fetch

ID: Instruction decode/
register file read

EX: Execute/
address calculation

MEM: Memory access

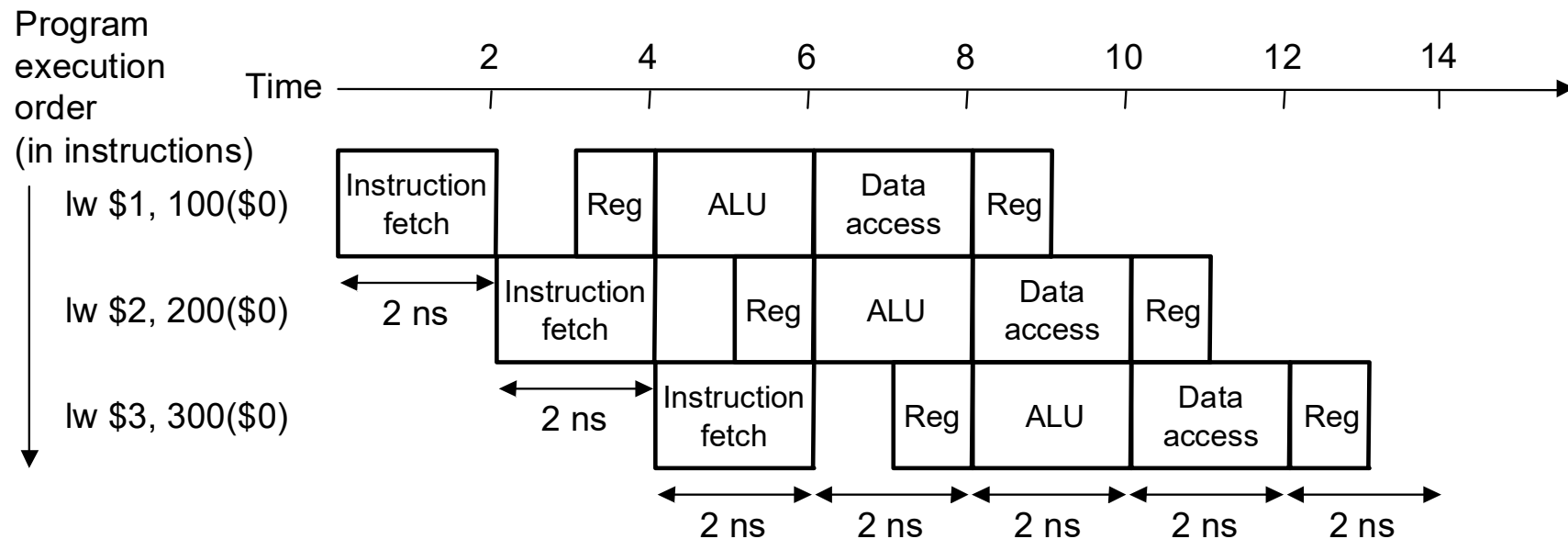
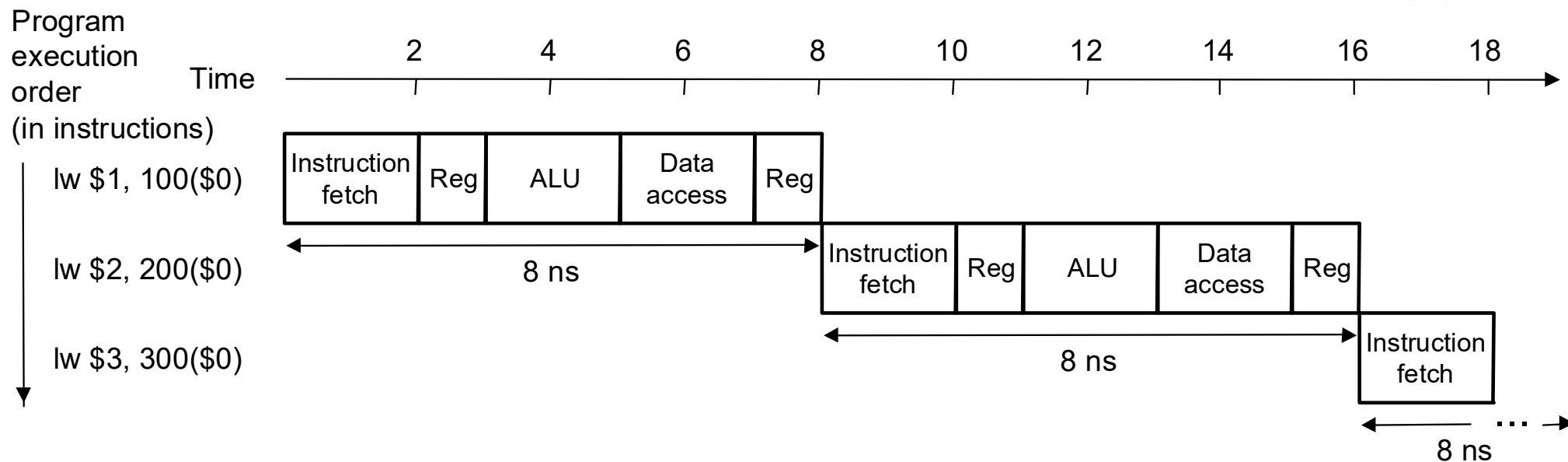
WB: Write back



Is this the correct partitioning?

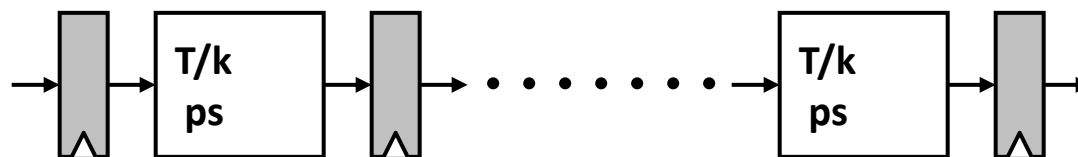
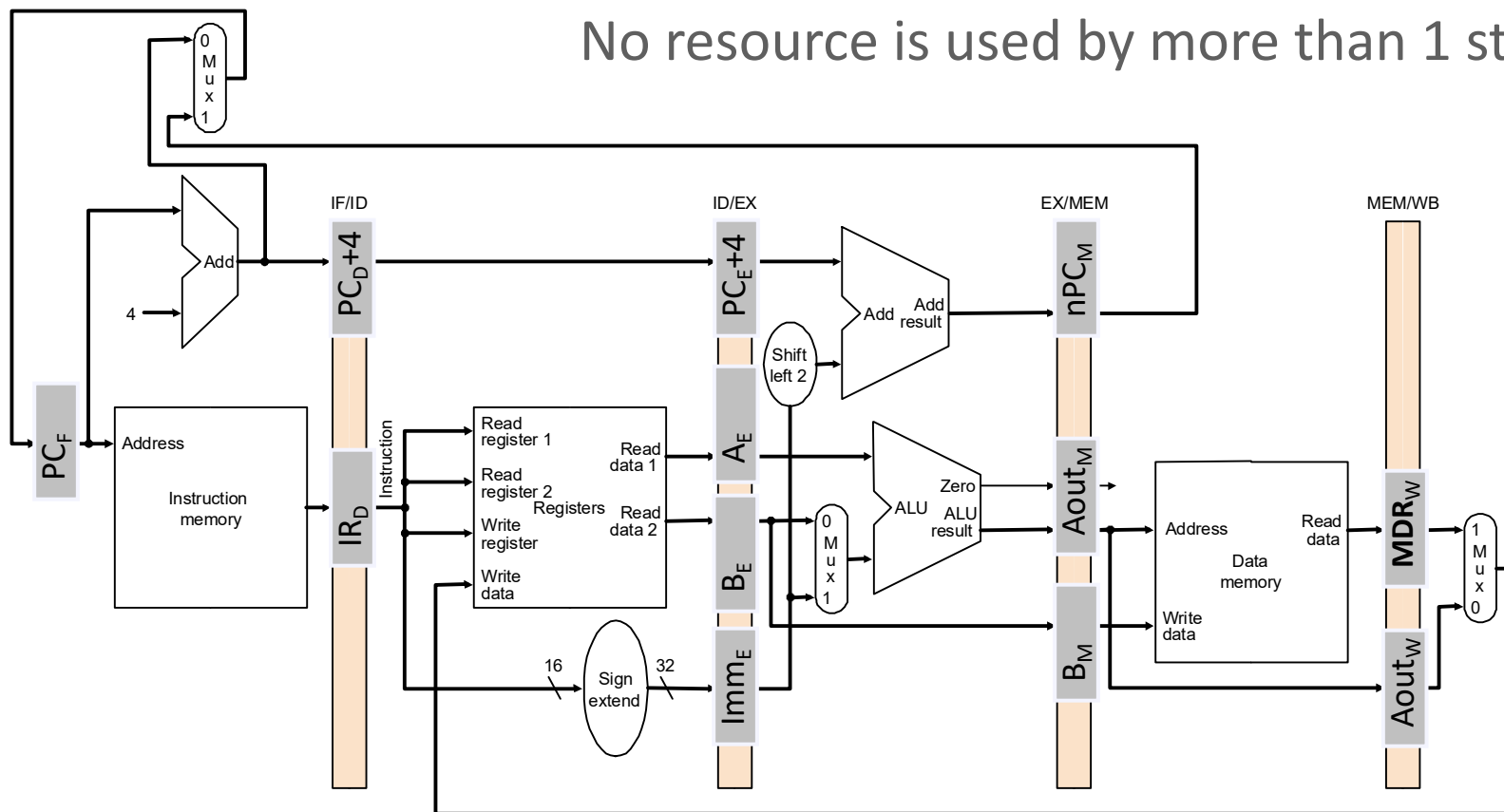
Why not 4 or 6 stages? Why not different boundaries?

指令流水线吞吐率

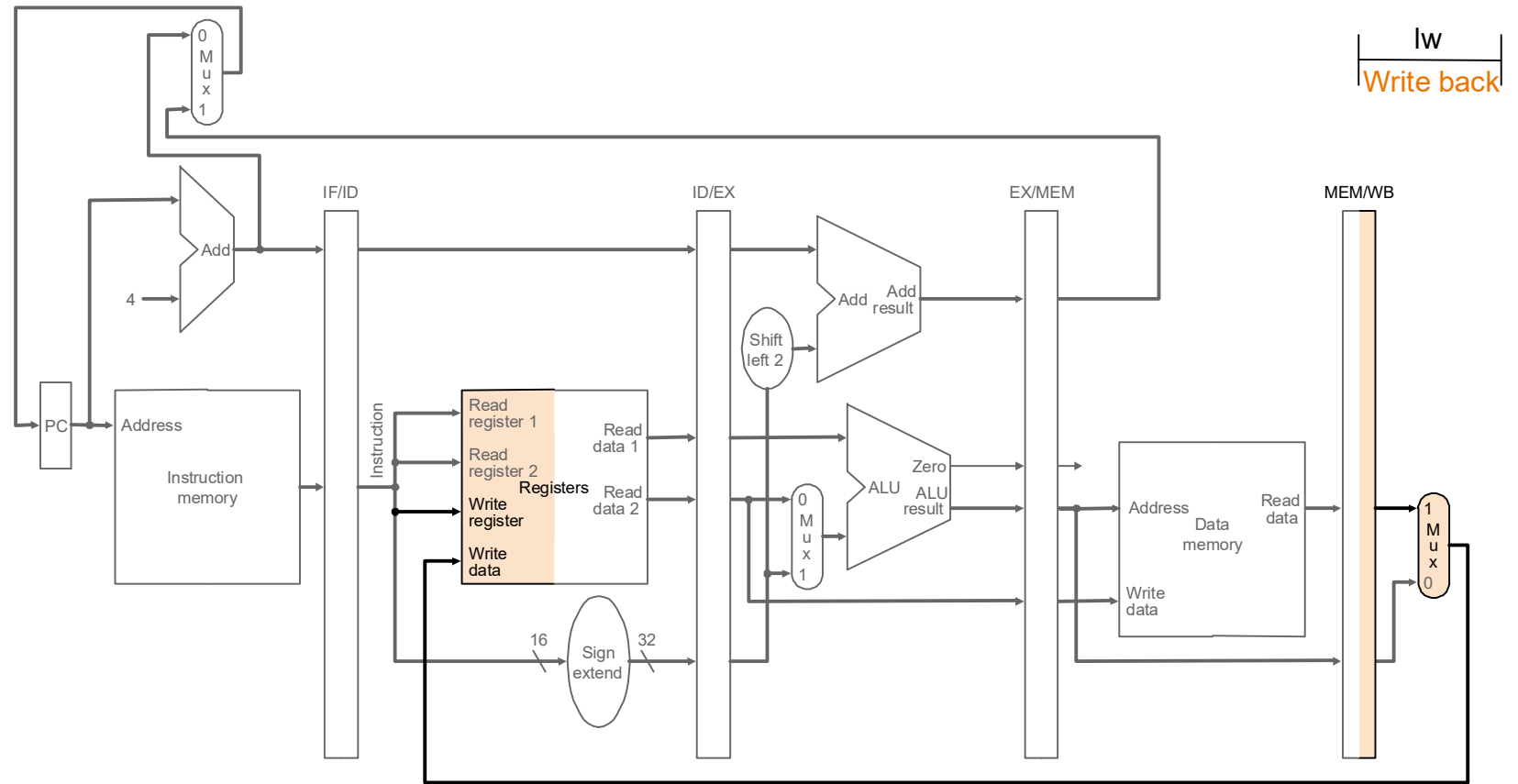


流水线操作示例

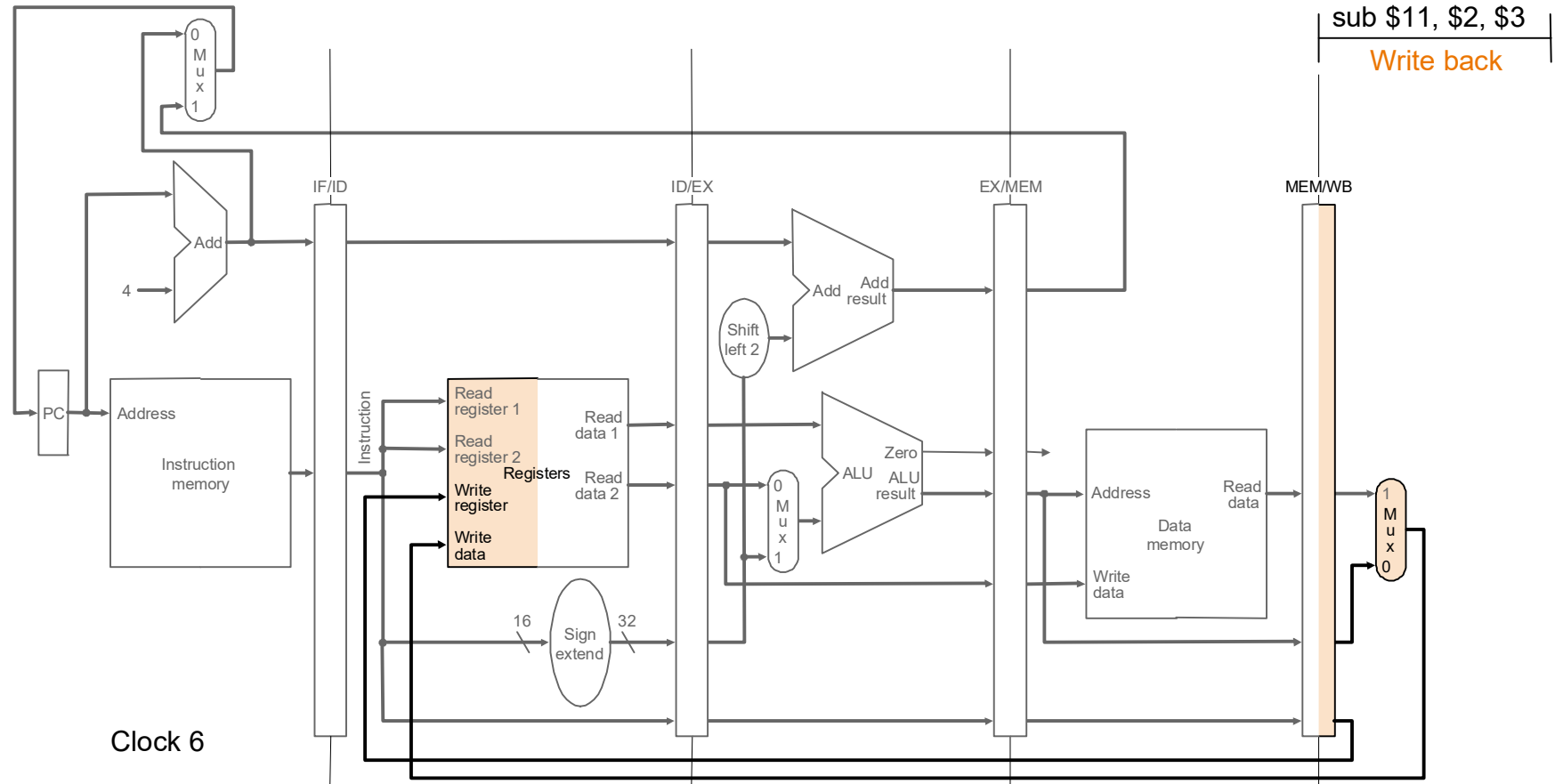
No resource is used by more than 1 stage!



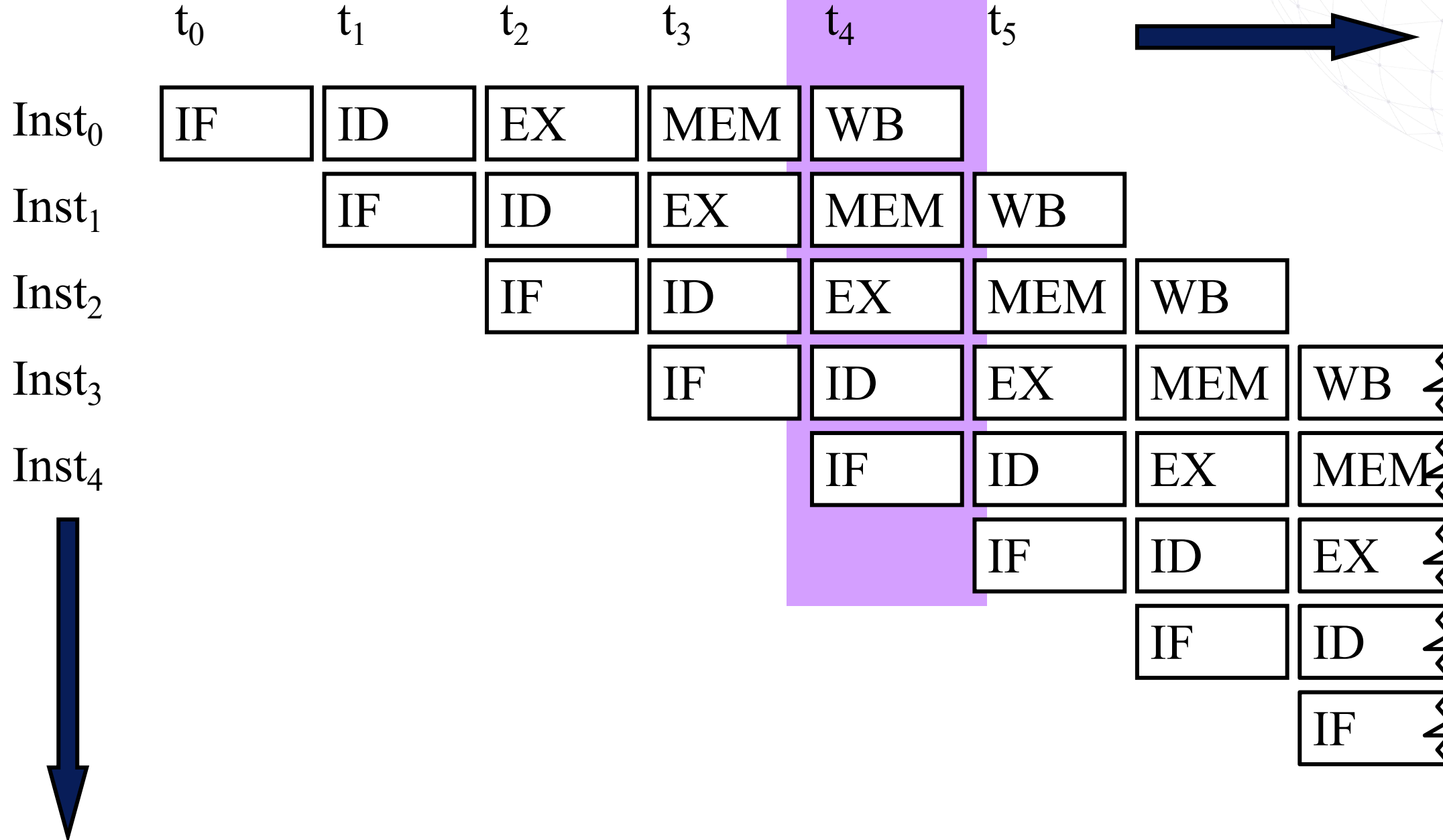
流水线操作示例



流水线操作示例



流水线操作：操作视图



流水线操作：资源视图

	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
IF	I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈	I ₉	I ₁₀
ID		I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈	I ₉
EX			I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈
MEM				I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇
WB					I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆

流水线的启示

- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Unbalanced lengths of pipe stages reduces speedup
- Time to “**fill**” pipeline and time to “**drain**” it reduces speedup

其他定义

- Pipe stage or pipe segment
 - A decomposable unit of the fetch-decode-execute paradigm
- Pipeline depth
 - Number of stages in a pipeline
- Machine cycle
 - Clock cycle time
- Latch
 - Per phase/stage local information storage unit

流水线设计问题

- Balance the length of each pipeline stage

$$\text{Throughput} = \frac{\text{Depth of the pipeline}}{\text{Time per instruction on unpipelined machine}}$$

- Problems
 - Usually, stages are not balanced
 - Pipelining overhead
 - Hazards (conflicts)
- Performance (throughput CPU performance equation)
 - Decrease of the CPI
 - Decrease of cycle time

第一和第二指令周期

- Instruction fetch (IF)

$IR \leftarrow \text{Mem}[PC];$

$NPC \leftarrow PC + 4$

- Instruction decode & register fetch (ID)

$A \leftarrow \text{Regs}[IR_{6..10}];$

$B \leftarrow \text{Regs}[IR_{11..15}];$

$\text{Imm} \leftarrow ((IR_{16})^{16} \# \# IR_{16..31})$

第三指令周期

- Execution & effective address (EX)
 - Memory reference
 - $\text{ALUOutput} \leftarrow A + \text{Imm}$
 - Register - Register ALU instruction
 - $\text{ALUOutput} \leftarrow A \text{ func } B$
 - Register - Immediate ALU instruction
 - $\text{ALUOutput} \leftarrow A \text{ op Imm}$
 - Branch
 - $\text{ALUOutput} \leftarrow \text{NPC} + \text{Imm}; \text{Cond} \leftarrow (A \text{ op } 0)$

第四指令周期

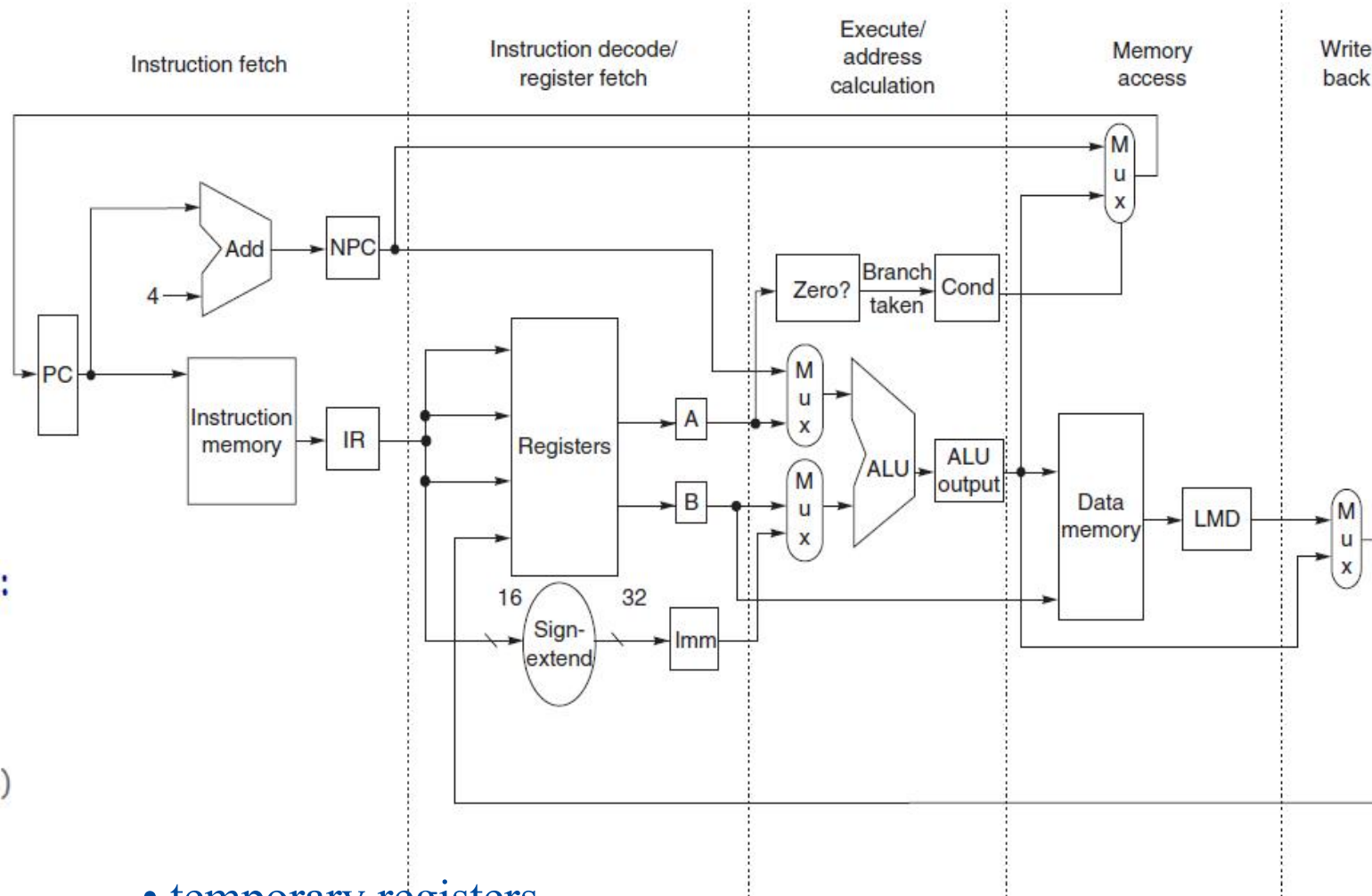
- Memory access & branch completion (MEM)
 - Memory reference
 - $PC \leftarrow NPC$
 - $LMD \leftarrow Mem[ALUOutput]$ (load)
 - $Mem[ALUOutput] \leftarrow B$ (store)
 - Branch
 - if (cond) $PC \leftarrow ALUOutput$; else $PC \leftarrow NPC$

第五指令周期

- Write-back (WB)
 - Register - register ALU instruction
 - $\text{Regs}[\text{IR}_{16..20}] \leftarrow \text{ALUOutput}$
 - Register - immediate ALU instruction
 - $\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{ALUOutput}$
 - Load instruction
 - $\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{LMD}$

MIPS 的简单实现

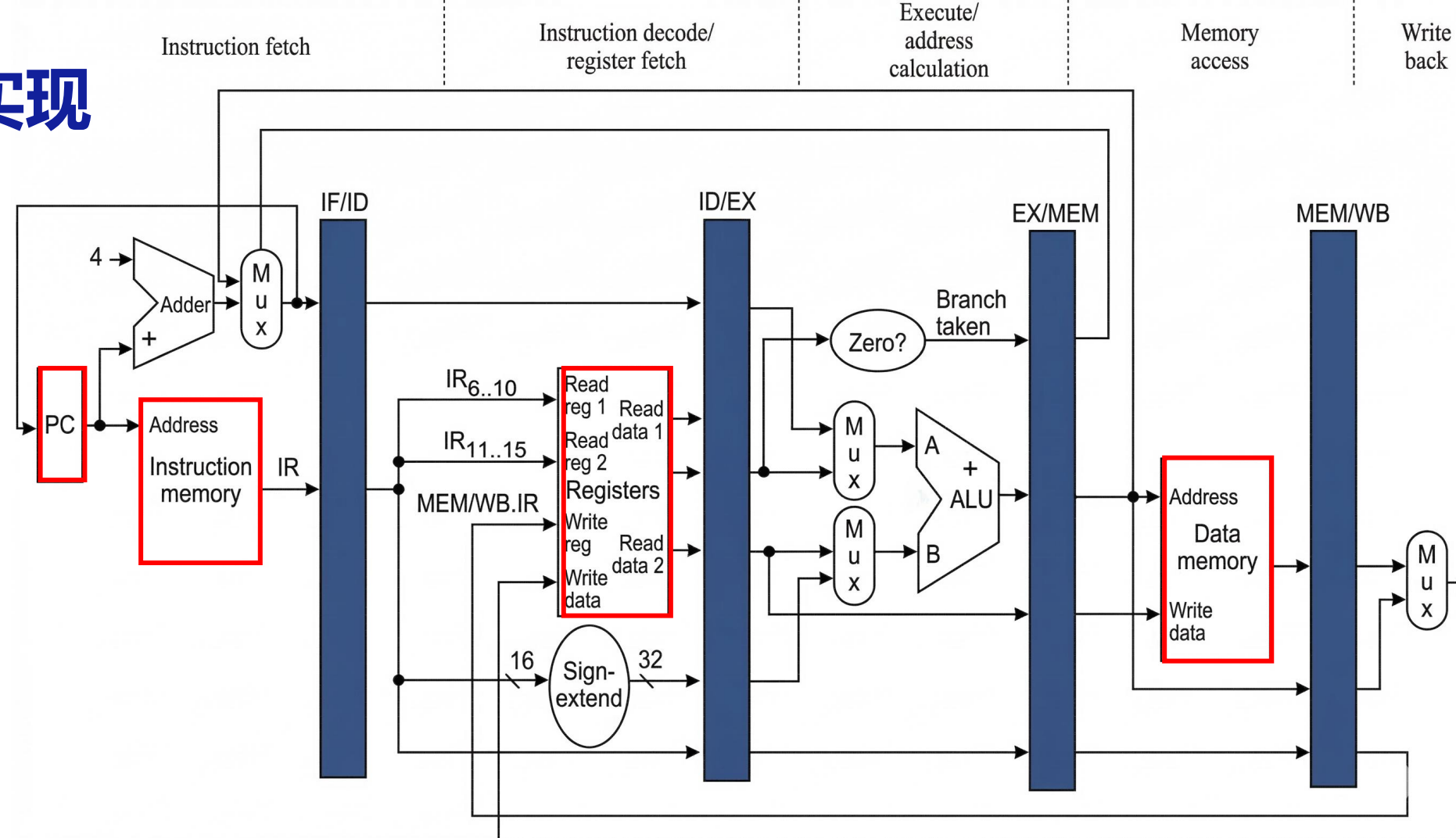
1. $IR \leftarrow Mem[PC];$
 $NPC \leftarrow PC + 4;$
2. $A \leftarrow Regs[rs];$
 $B \leftarrow Regs[rt];$
 $Imm \leftarrow signExt($
 $IR_{16})$
 - fixed-field decoding
3. one of the followings:
 - $ALUOutput \leftarrow A + Imm$
 - $ALUOutput \leftarrow func(A, B)$
 - $ALUOutput \leftarrow A \text{ op } Imm$
 - $Cond \leftarrow (A == zero) \text{ and }$
 $ALUOutput \leftarrow NPC + (Imm \ll 2)$
4. $PC \leftarrow MUX(Cond,$
 $ALUOutput, NPC)$
 - $LMD \leftarrow Mem[ALUOutput] \text{ or }$
 - $Mem[ALUOutput] \leftarrow B$
5. one of the followings:
 - $Regs[rt] \leftarrow LMD$
 - $Regs[rd] \leftarrow ALUOutput$
 - $Regs[rt] \leftarrow ALUOutput$



- temporary registers
 - hold values between clock cycles for an instruction
- state elements (“visible part of the state”)
 - hold values between successive instructions
- control logic (FSM or microcode controller)
 - (not illustrated in the diagram above)

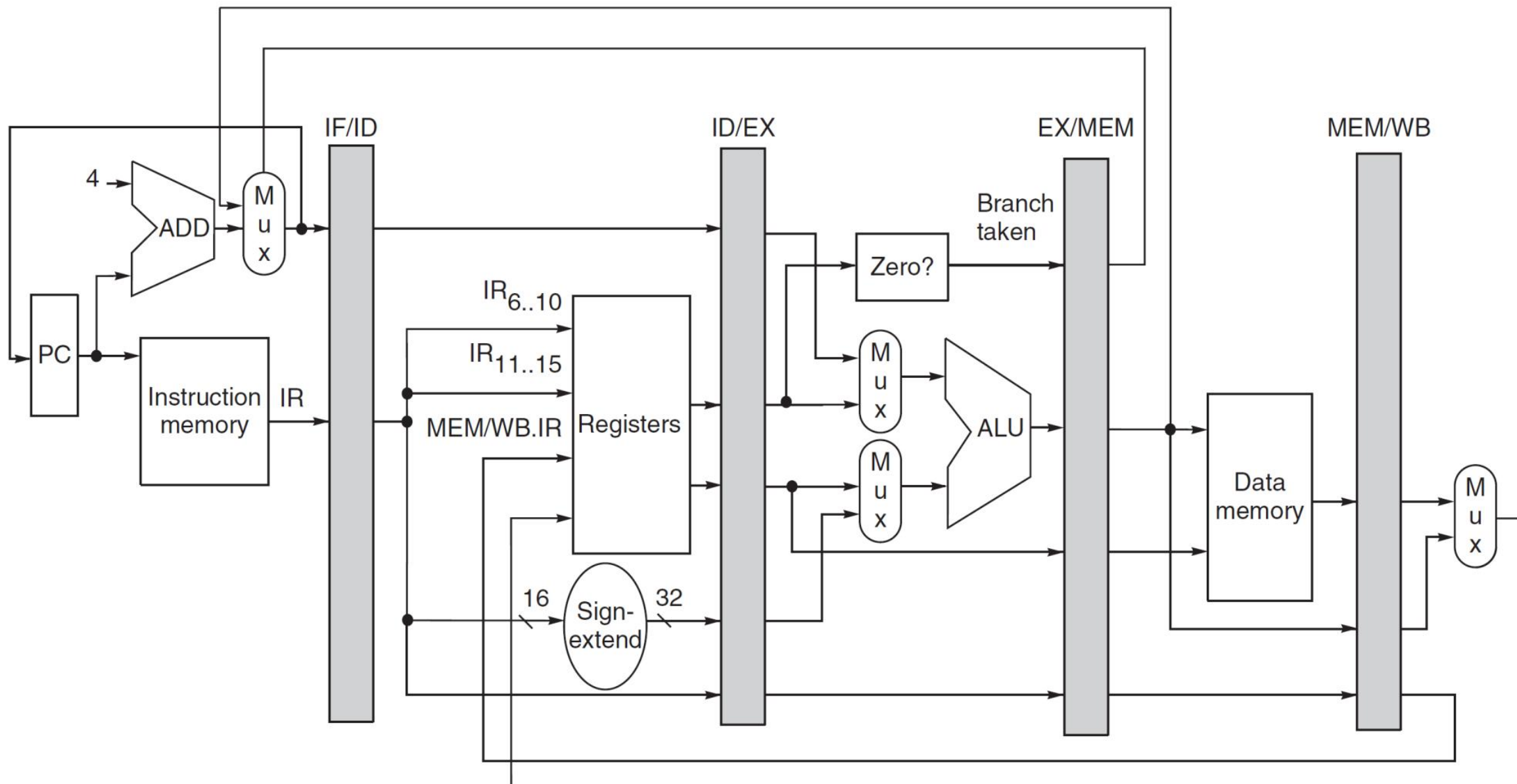
MIPS 流水线实现

- Apparently easy
 - each clock cycle becomes a pipe stage
 - temporary registers become pipe registers
 - new instruction issued at each clock cycle
- Result propagation
 - register value to be stored is read during ID and used in MEM
 - ALU result computed during EX (or loaded during MEM) and store in WB



- pipeline registers
 - hold values between clock cycles for an instruction
 - prevent interference (edge-triggered flip-flops)
- state elements (“visible part of the state”)
 - hold values between successive instructions

MIPS 流水线实现



MIPS 流水线实现

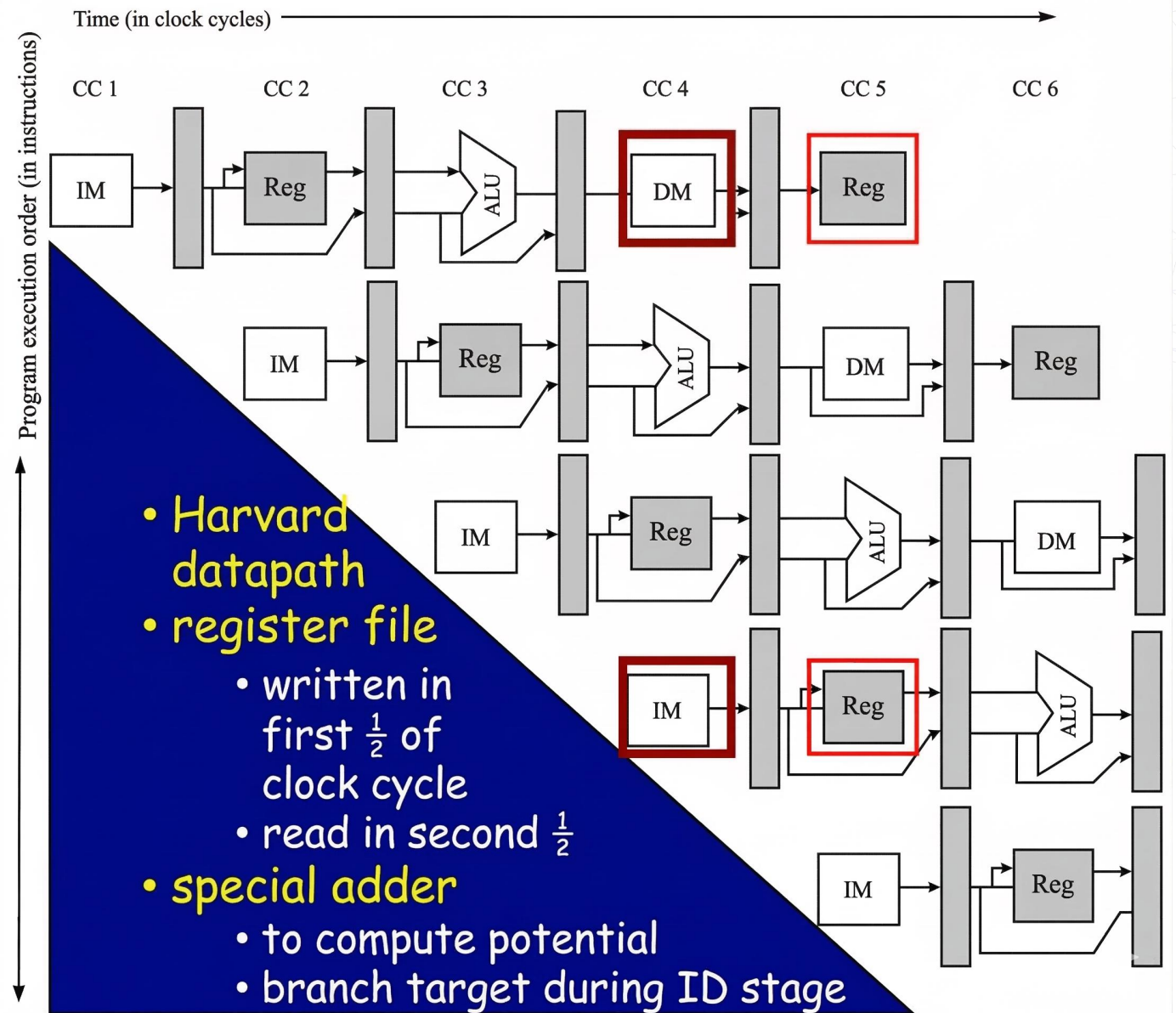
lw R10, 20(R1)

sub R11, R2, R3

add R12, R3, R4

lw R13, 24(R1)

add R14, R5, R6



Unlike some other speedup techniques, pipelining is fundamentally transparent to the programmer

MIPS 流水线：各阶段事件

Stage	ALU Instruction	Load/Store Instruction	Branch Instruction
IF	$IF/ID.IR \leftarrow Mem[PC];$ $IF/ID.NPC \leftarrow \text{if } ((EX/MEM.opc == \text{branch}) \ \& \ EX/MEM.cond) \ EX/MEM.AluOutput \text{ else } PC+4;$		
ID	$ID/EX.A \leftarrow Regs[IF/ID.IR[rs]]; \quad ID/EX.B \leftarrow Regs[IF/ID.IR[rt]];$ $ID/EX.NPC \leftarrow IF/ID.NPC; \quad ID/EX.IR \leftarrow IF/ID.IR;$ $ID/EX.Imm \leftarrow \text{sign-extend}(IF/ID.IR[\text{immediate field}])$		
EX	$EX/MEM.IR \leftarrow ID/EX.IR;$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.B \ \text{or}$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.Imm$	$EX/MEM.IR \leftarrow ID/EX.IR;$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.Imm$	$EX/MEM.ALUOutput \leftarrow ID/EX.NPC + (ID/EX.Imm \ll 2)$ $EX/MEM.Cond \leftarrow ID/EX.A == 0)$
MEM	$MEM/WB.IR \leftarrow EX/MEM.IR$ $MEM/WB.ALUOutput \leftarrow EX/MEM.AluOutput$	$MEM/WB.IR \leftarrow EX/MEM.IR$ $MEM/WB.LMD \leftarrow Mem[EX/MEM.AluOutput]$ <i>or</i> $Mem[EX/MEM.AluOutput] \leftarrow EX/MEM.B$	
WB	$Regs[MEM/WB.IR[rd]] \leftarrow MEM/WB.AluOutput \ \text{or}$ $Regs[MEM/WB.IR[rt]] \leftarrow MEM/WB.AluOutput$	<i>for load only:</i> $Regs[MEM/WB.IR[rt]] \leftarrow MEM/WB.LMD$	

MIPS 简化流水线的特性

- ISA encoding: all instructions have same length
 - balanced separation of fetch and decode stages
- ISA encoding: few instructions formats with the source register fields in the same position
 - symmetry allows reading of register file and decoding of instruction simultaneously during the second stage (fixed-field decoding)
- Memory accessed only by load/store instructions
 - memory address is “pre-computed” during the execution stage for the following stage (no operation involves operands in memory)
- Each operation writes at most on result (and near the end of the pipeline)
 - simplifies forwarding (useful for handling data hazards)
- All instructions change an entire register, memory access does not require realignment...
 - simpler instructions can be decomposed in steps that can be performed each in a single pipeline stage

基础流水线

Instr #	Clock number								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
$i + 1$		IF	ID	EX	MEM	WB			
$i + 2$			IF	ID	EX	MEM	WB		
$i + 3$				IF	ID	EX	MEM	WB	
$i + 4$					IF	ID	EX	MEM	WB

第一批流水线计算机

- IBM 7030 “Stretch”
 - considered one of the first general-purpose pipelined computer (late 1950s)
 - pipeline overlapping fetch, decode, execute stages
- CDC 6600
 - Control Data Corp. (Seymour Cray)
 - the first supercomputer (1964)
 - interaction between ISA and pipeline HW was well understood
 - ISA was kept simple on purpose
- IBM 360
 - more complex pipeline with Tomasulo’s Algorithm (1964)



MIPS 流水线：基本性能问题

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

- **Pipelining**

- improve **CPU Throughput**
(reciprocal of CPU Time)
- slightly increases execution time
- result: program run faster!

$$[\text{time per instruction}]_{\text{pipelined}} = \frac{[\text{time per instruction}]_{\text{nonpipelined}}}{\text{number of pipeline stages}}$$

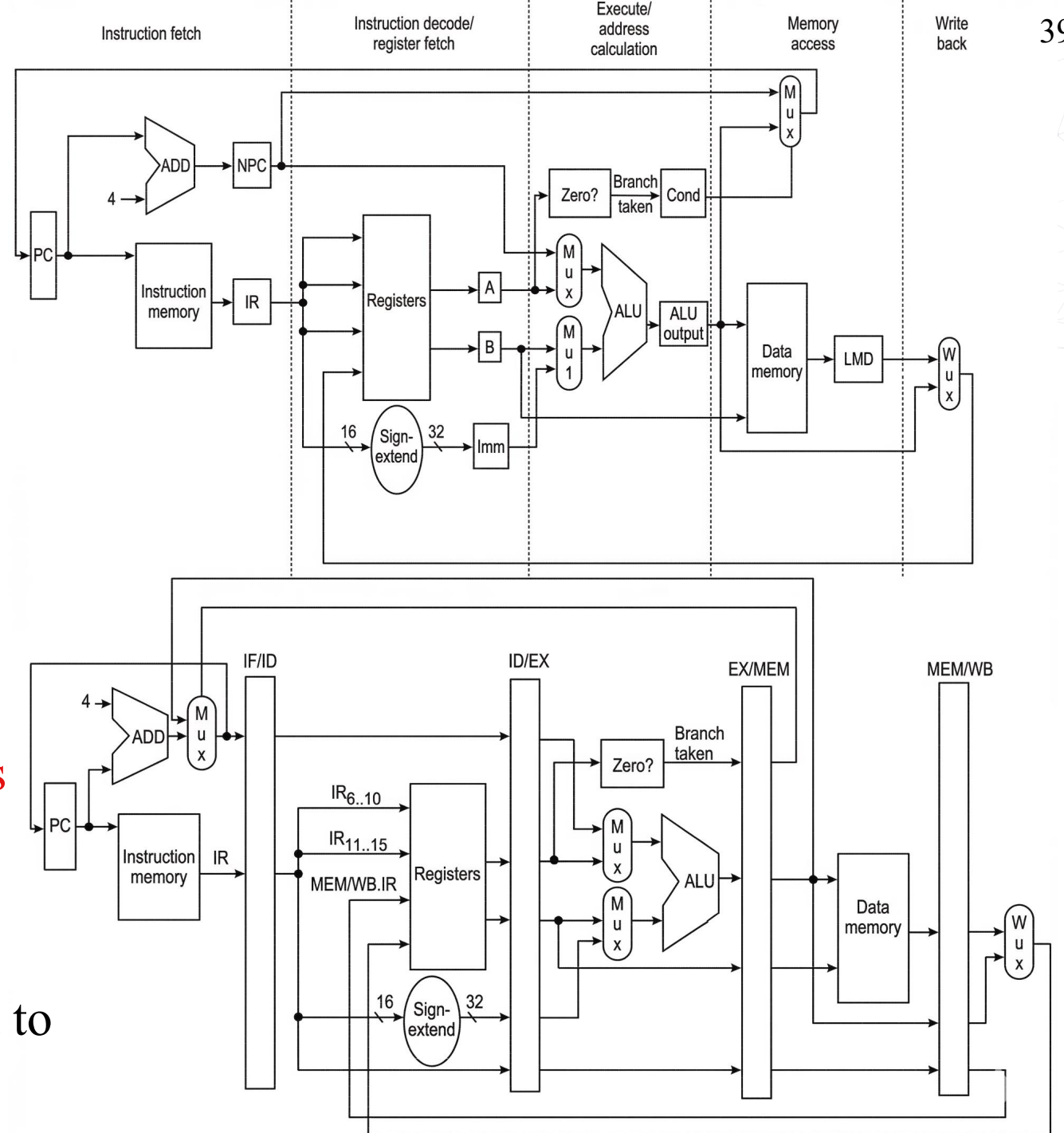
- **Pipelining can be seen as either**

- decreasing the **CPI** of a multi-cycle un-pipelined implementation
- decreasing the **CCT** of a single-cycle un-pipelined implementation

	IC	CPI	CCT
Program	√		
Compiler	√		
ISA	√	√	
HW organization		√	√
HW technology			√

示例： 流水线加速比 vs. 多周期非流水线实现

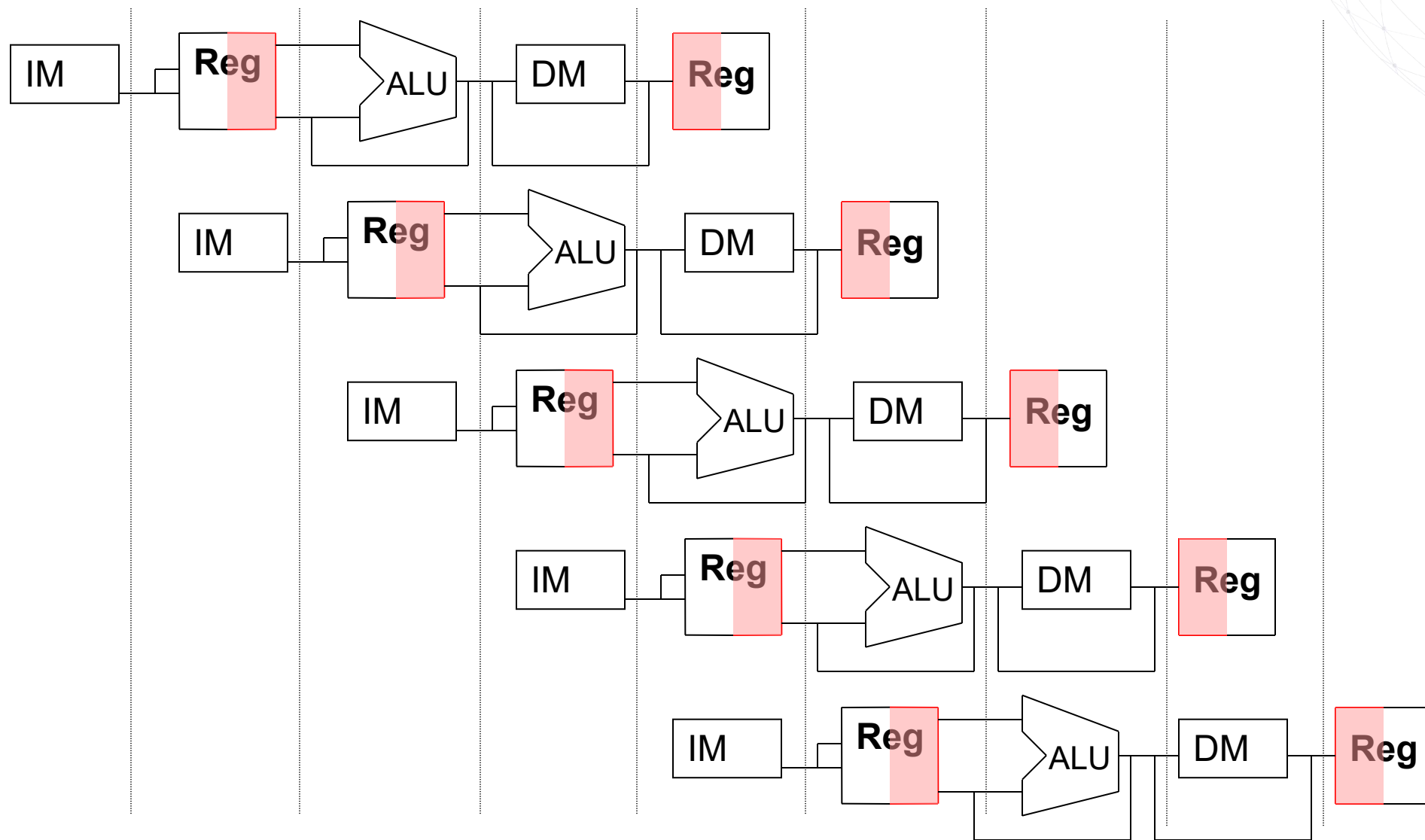
- Instruction CPI
 - 4 cycles (branches and stores)
 - 5 cycles (other instructions)
- Average CPI = 4.75 assuming
 - 15% branches
 - 10% stores
- Average Instruction ExecTime is 4.75ns, assuming CCT = 1ns
- Ideal CPI = 1 (almost always)
- Average Instruction ExecTime is 1.2ns with CCT = 1ns and clock overhead = 0.2 ns
- Speedup is 3.96x for ideal pipeline
 - as we'll see soon, in reality we need to sum the pipeline stalled clock cycles per



性能限制

- Imbalance among pipe stages
 - limits cycle time to slowest stage
- Pipelining overhead
 - Pipeline register delay
 - Clock skew
- Clock cycle $>$ clock skew + latch overhead
- Hazards

流水线资源



流水线冒险及其分类

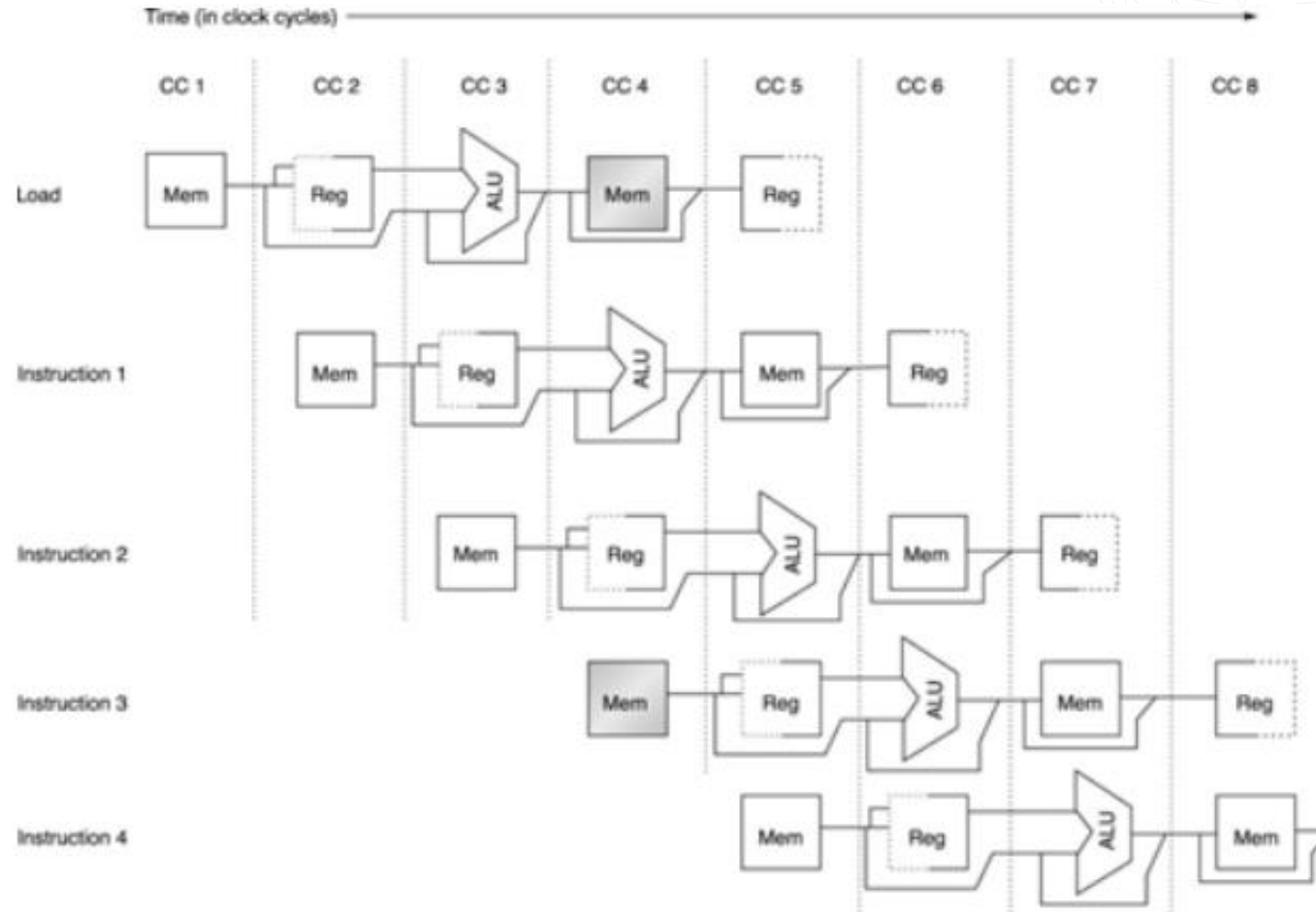
- Pipeline hazard situation
 - “the next instruction cannot execute in the following clock cycle”
- Pipeline hazards come in three different flavors
 - Structural Hazards
 - arise from resource conflict: the HW cannot support all possible instruction combinations simultaneously in overlapped execution
 - Data Hazards
 - arise when an instruction depends on the result of a previous instruction in a way exposed by the pipeline overlapped execution
 - Control Hazards
 - arise from the pipelining of branches, jumps...
- Hazards may force pipeline stalling
 - Instructions issued after the stalled one must stall also (and fetching is stalled) while all those issued earlier must proceed

结构冒险

- Overlapped execution of instructions:
 - Pipelining of functional units
 - Duplication of resources
- Structural Hazard
 - When the pipeline can not accommodate some combination of instructions
- Consequences
 - Stall
 - Increase of CPI from its ideal value (1)

结构冒险：示例

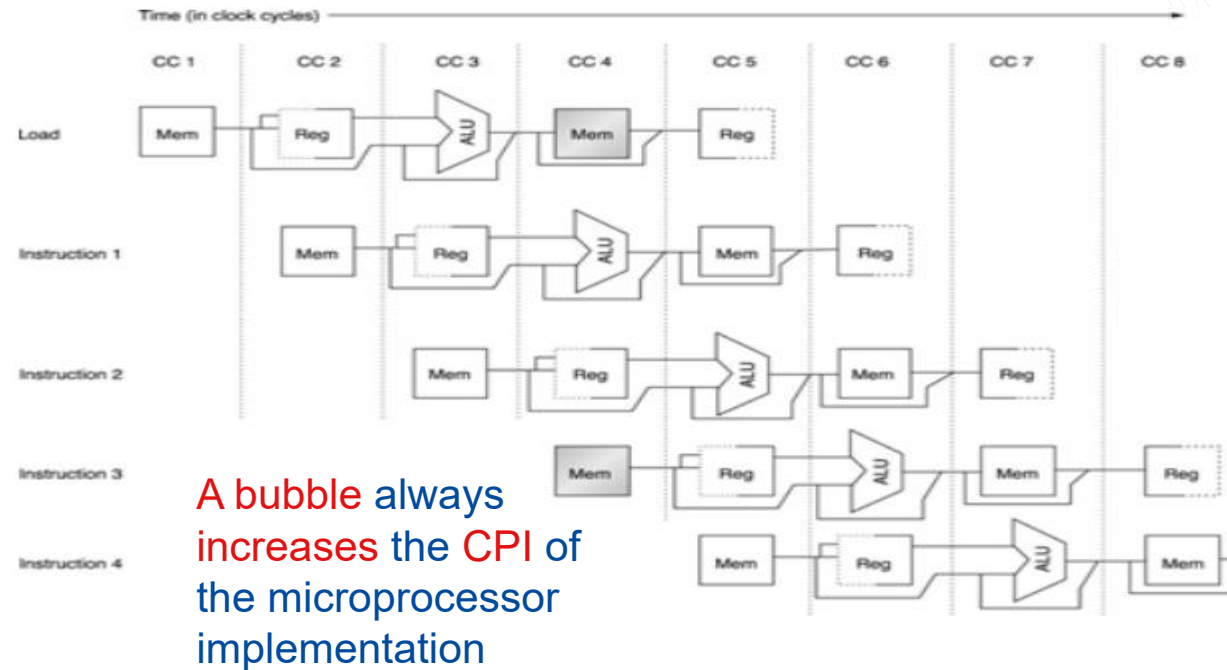
- Structural hazards are due to resource constraints:
 - some resources are not duplicated enough to allow all combination of instructions in the pipeline to execute
 - e.g., the architecture is not Harvard-like
 - a functional unit is not fully pipelined
 - e.g., an instruction takes more than one clock cycle to go through it



The ultimate reason for the presence of structural hazards is the designers' attempt to reduce implementation costs

流水线停顿（空泡）：从统一单端口存储器进行加载 / 取指的示例

- Structural hazard causes a bubble
 - it floats through the pipeline taking space but carrying no useful information



instruction	1	2	3	4	5	6	7	8	9
LW R10, 20(R1)	IF	ID	EX	MEM	WB				
SUB R11, R2, R3		IF	ID	EX	MEM	WB			
ADD R12, R3, R4			IF	ID	EX	MEM	WB		
STALL									
ADD R14, R5, R6					IF	ID	EX	MEM	WB

停顿对流水线实现性能的影响

$$\text{speedupFromPipelining} = \frac{[CPI]_{\text{nonpipelined}} \times [CCT]_{\text{nonpipelined}}}{[CPI]_{\text{pipelined}} \times [CCT]_{\text{pipelined}}}$$

$$\begin{aligned} [CPI]_{\text{pipelined}} &= \text{idealCPI} + \text{pipelineStallCyclesPerInstruction} \\ &= 1 + \text{pipelineStallCyclesPerInstruction} \end{aligned}$$

- Special case: ignoring the pipeline clock-period overhead and assume that
 - the pipeline stages are perfectly balanced
 - all instructions take the same number of cycles in the non-pipelined implementation (equal to the pipeline depth)

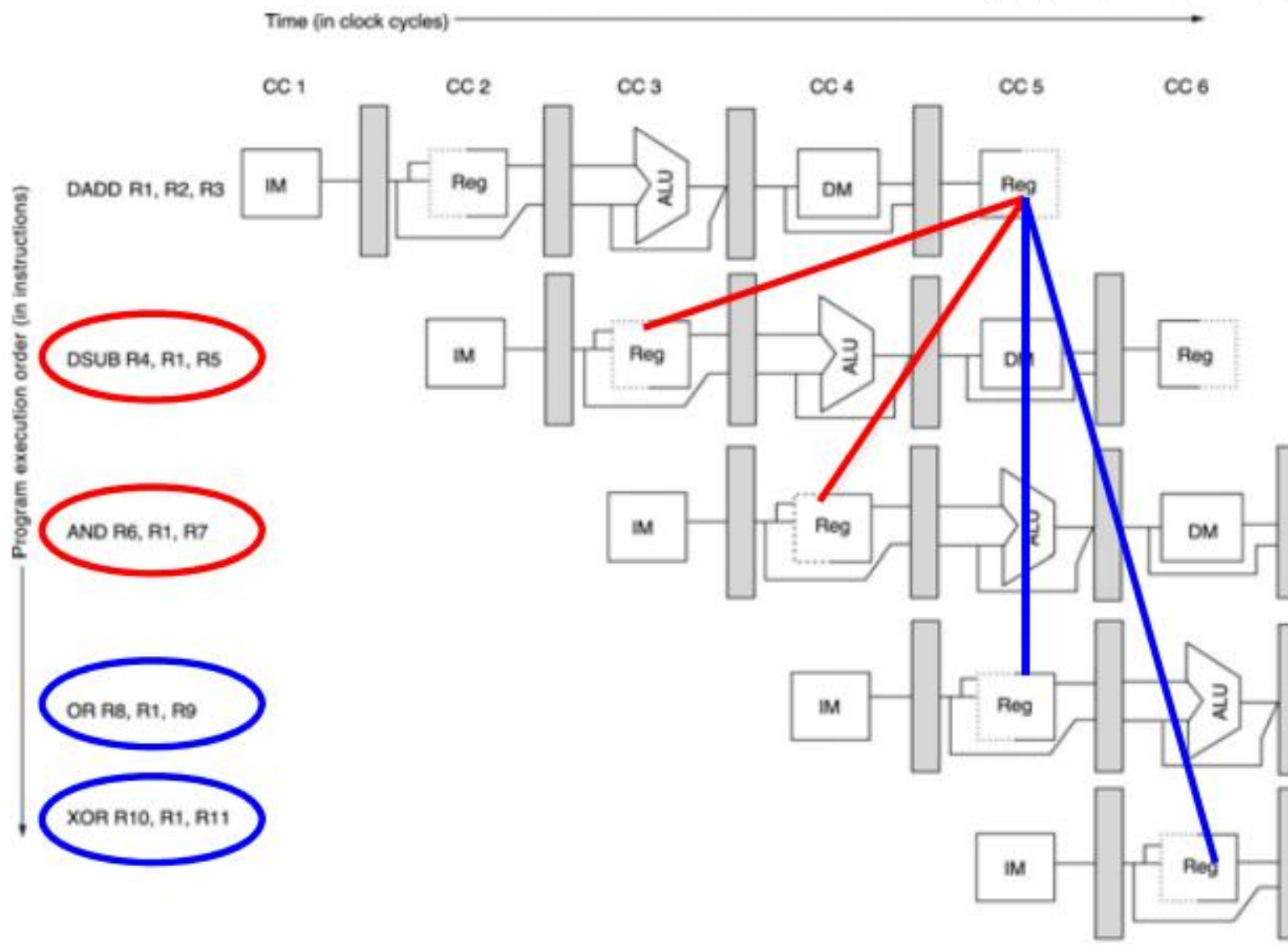
$$\text{speedupFromPipelining} = \frac{\text{pipelineDepth}}{1 + \text{pipelineStallCyclesPerInstruction}}$$

停顿对流水线实现性能的影响 - 示例

- Implementation without structural hazards
 - ideal CPI = 1
 - CCT = 1ns
- Implementation with the “load/store” structural hazard
 - CCT = 900ps
- Suppose that
 - 40% of the executed instructions are loads or stores
- Which implementation is faster?
 - $(\text{averageInstructionTime})_{\text{noHaz}} = 1 * 1 = 1$
 - $(\text{averageInstructionTime})_{\text{haz}} = (1 + 0.4 * 1) * 0.9 = 1.26$
 - implementation without structural hazard is 1.26 times faster than the other

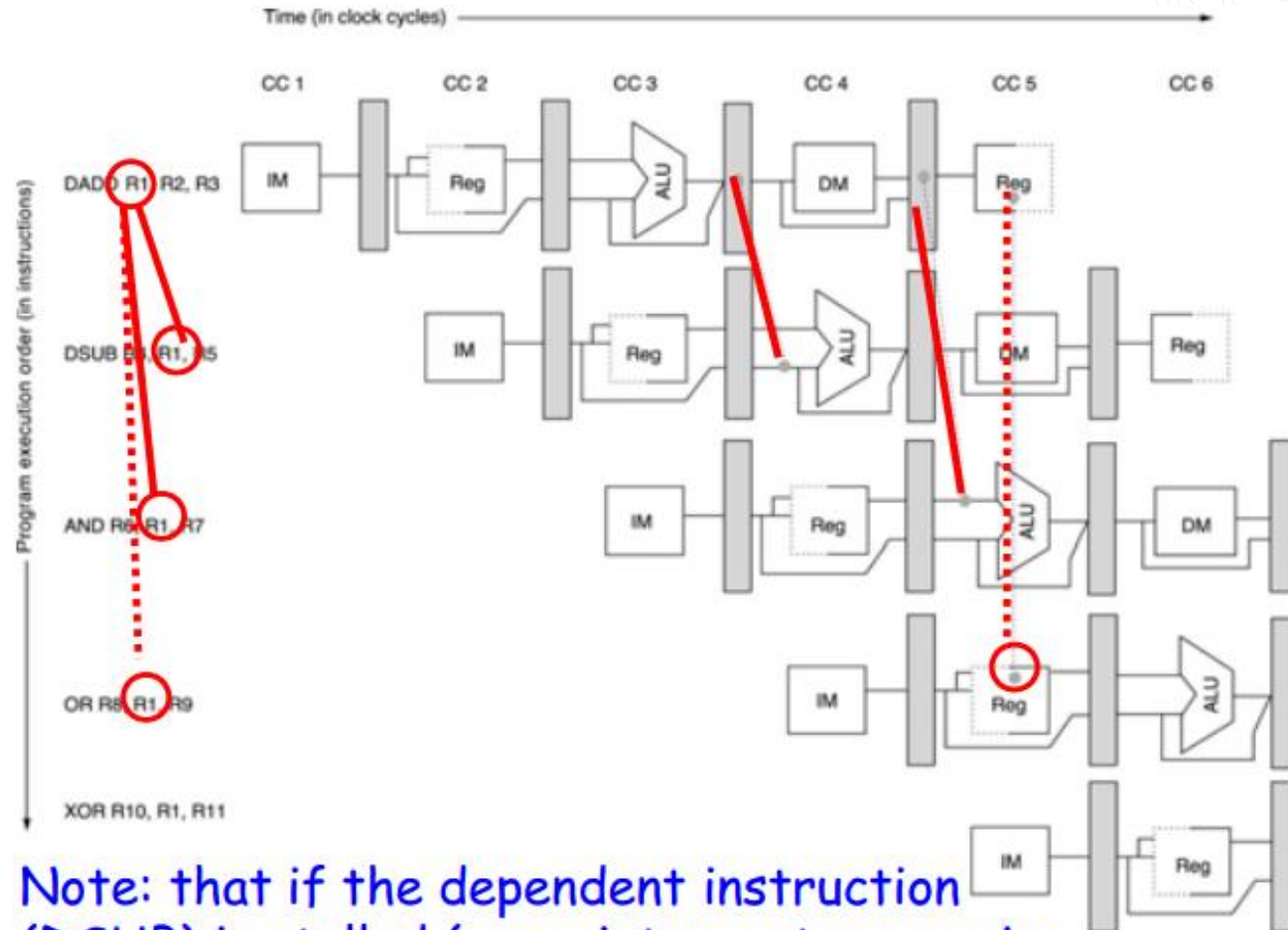
数据冒险

- Which value do these instructions read from R1 ?
 - Unless we take care of the data hazard the value read is not even deterministic!
- This data hazard is called RAW (read after write)
 - the name refers to the expected execution flow



减少数据冒险：数据转发（旁路技术）

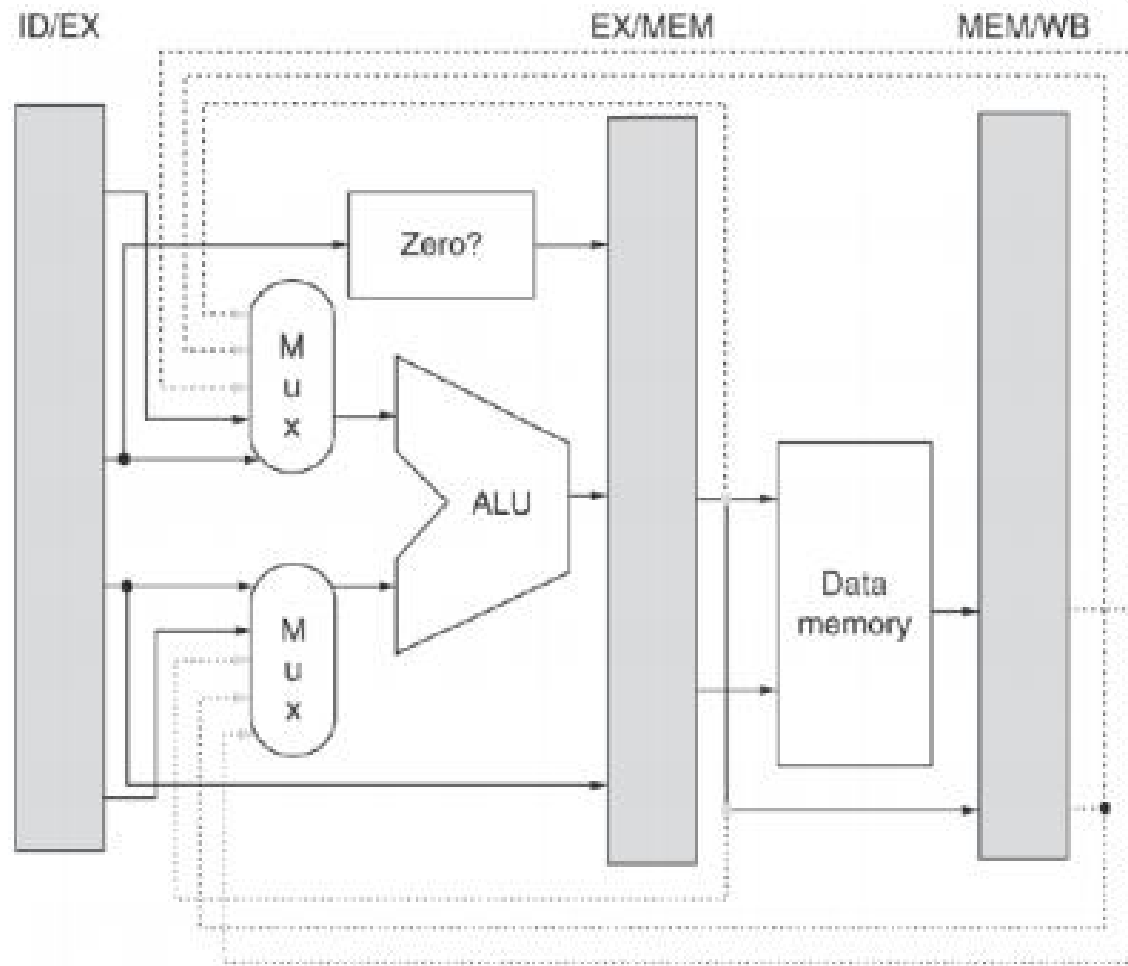
- An efficient technique based on a simple idea
 - subsequent instructions only need the value in R1 after this has been computed!
- ALU result is always fed back to the ALU input from both EX/MEM and MEM/WB



Note: that if the dependent instruction (DSUB) is stalled (or an interrupt occurs in between the two instructions) then the forward path will not be activated

向 ALU 转发数据

- 3 inputs added to the MUX
- 3 bypass paths
 1. From ALU output at the end of EX
 2. From ALU output at the end of MEM stage
 3. The memory output at the end of the MEM stage



数据依赖

- Types of data dependences
 - **Flow dependence** (true data dependence – read after write)
 - **Output dependence** (write after write)
 - **Anti dependence** (write after read)
- Which ones cause stalls in a pipelined machine?
 - For all of them, we need to ensure semantics of the program are correct
 - Flow dependences always need to be obeyed because they constitute true dependence on a value
 - Anti and output dependences exist due to limited number of architectural registers
 - They are dependence on a name, not a value
 - We will later see what we can do about them

如何处理数据依赖

- Anti and output dependences are easier to handle
 - write to the destination in one stage and in program order
- Flow dependences are more interesting
- Five fundamental ways of handling flow dependences
 - **Detect and wait** until value is available in register file
 - **Detect and forward/bypass** data to dependent instruction
 - **Detect and eliminate** the dependence at the software level
 - No need for the hardware to detect dependence
 - **Predict** the needed value(s), execute “speculatively”, **and verify**
 - **Do something else** (fine-grained multithreading)
 - No need to detect

数据依赖类型

Flow dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$



Read-after-Write
(RAW)

Anti dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_1 \leftarrow r_4 \text{ op } r_5$



Write-after-Read
(WAR)

Output-dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$
 $r_3 \leftarrow r_6 \text{ op } r_7$



Write-after-Write
(WAW)

复习：带数据依赖的示例

Code Fragment

```
sub $2, $1, $3  
and $12, $2, $5  
or  $13, $6, $2  
add $14, $2, $2  
sw  $15, 100($2)
```

There are several **dependencies** in this new code fragment.

- The first instruction, SUB, stores a value into \$2.
- That register is used as a source in the rest of the instructions.

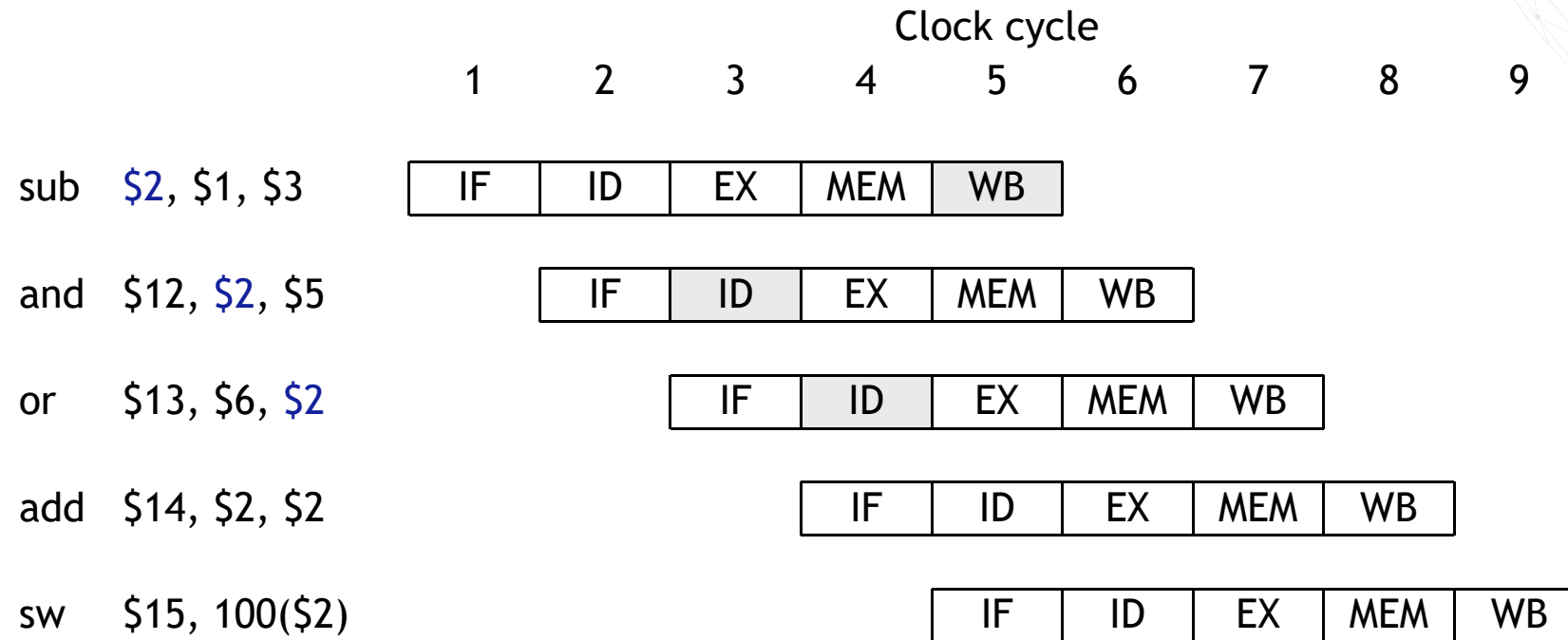
This is not a problem for the single-cycle and multicycle datapaths.

- Each instruction is executed completely before the next one begins.
- This ensures that instructions 2 through 5 above use the new value of \$2 (the sub result), just as we expect.



Question: How would this code sequence fare in our pipelined datapath?

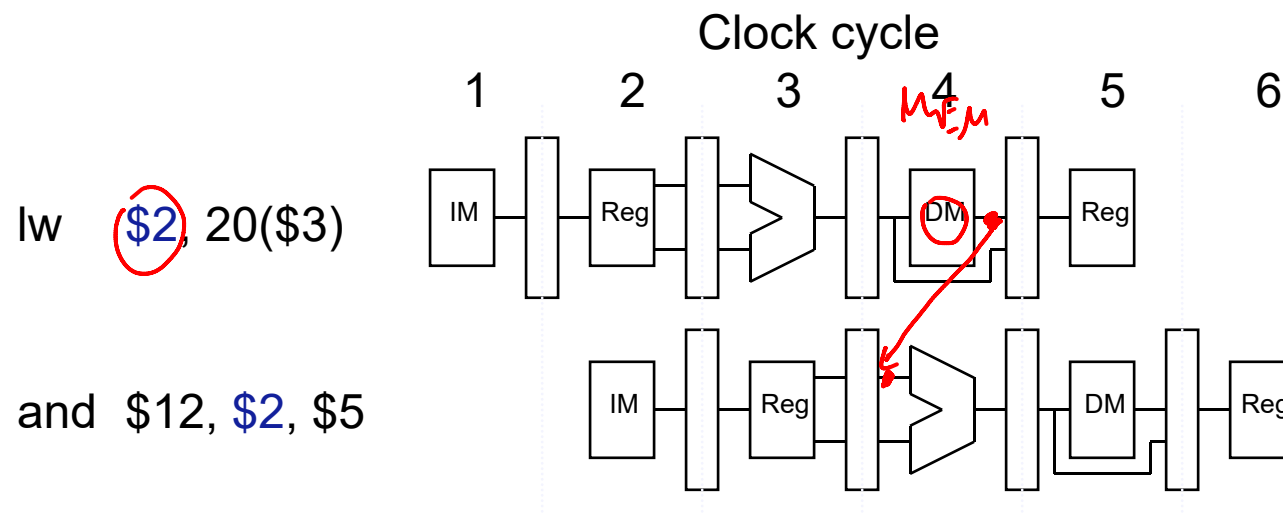
流水线图中的数据冒险



- The SUB instruction does not write to register \$2 until clock cycle 5. This causes two **data hazards** in our current pipelined datapath.
 - The AND reads register \$2 in cycle 3. Since SUB hasn't modified the register yet, this will be the *old* value of \$2, not the new one.
 - Similarly, the OR instruction uses register \$2 in cycle 4, again before it's actually updated by SUB.

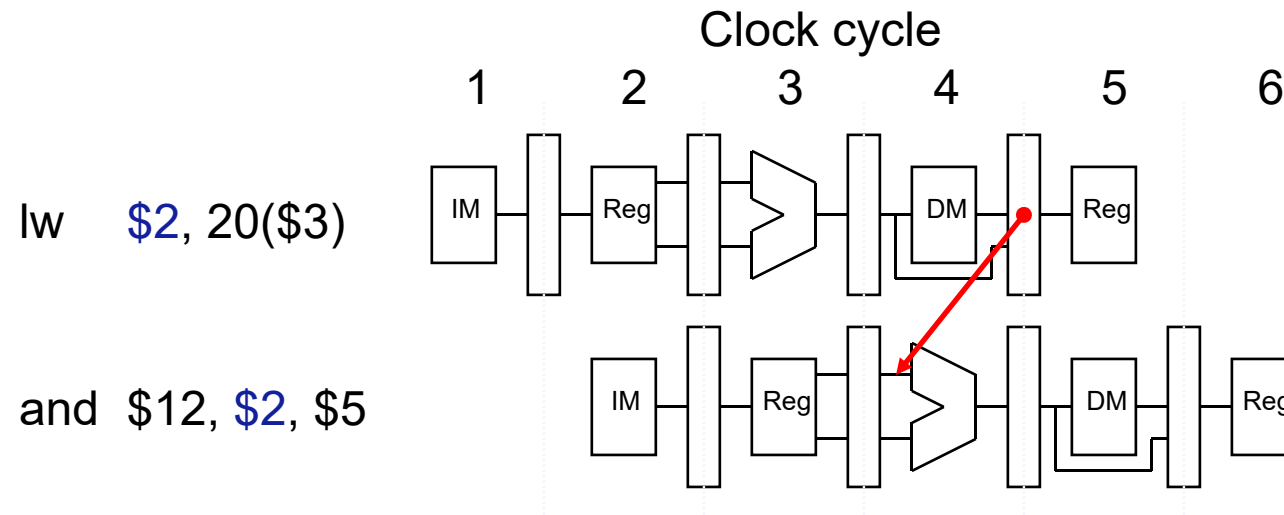
负载呢?

- Imagine if the first instruction in the example was LW instead of SUB.
 - How does this change the data hazard?



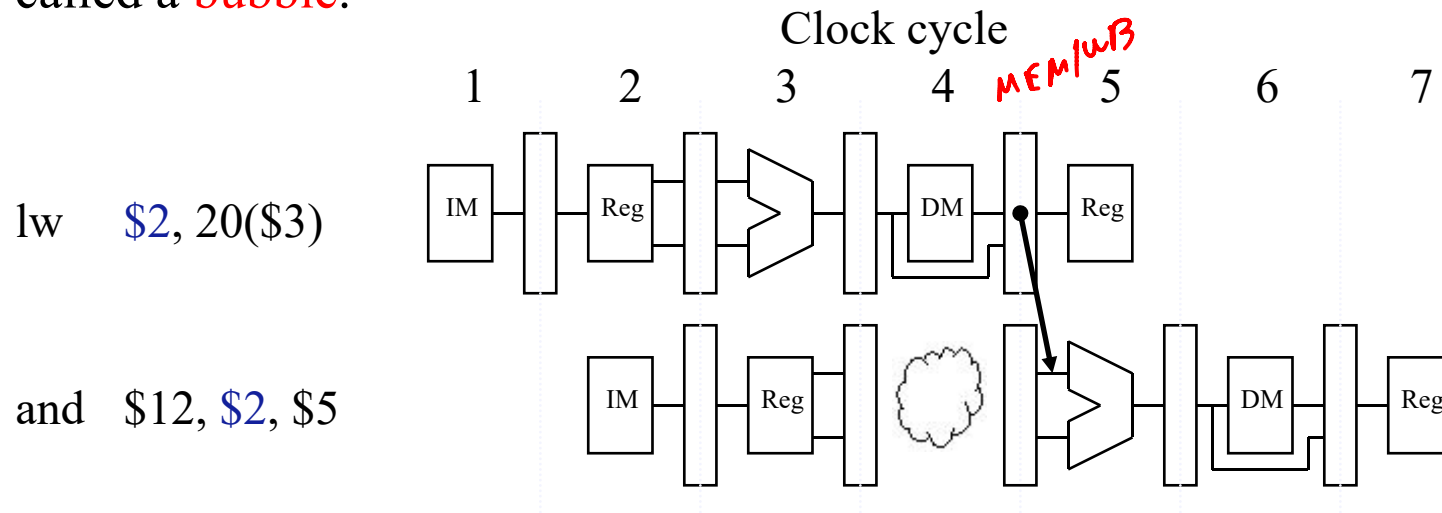
负载呢?

- Imagine if the first instruction in the example was LW instead of SUB.
 - The load data doesn't come from memory until the *end* of cycle 4.
 - But the AND needs that value at the *beginning* of the same cycle!
- This is a “true” data hazard—the data is not available when we need it.



停顿

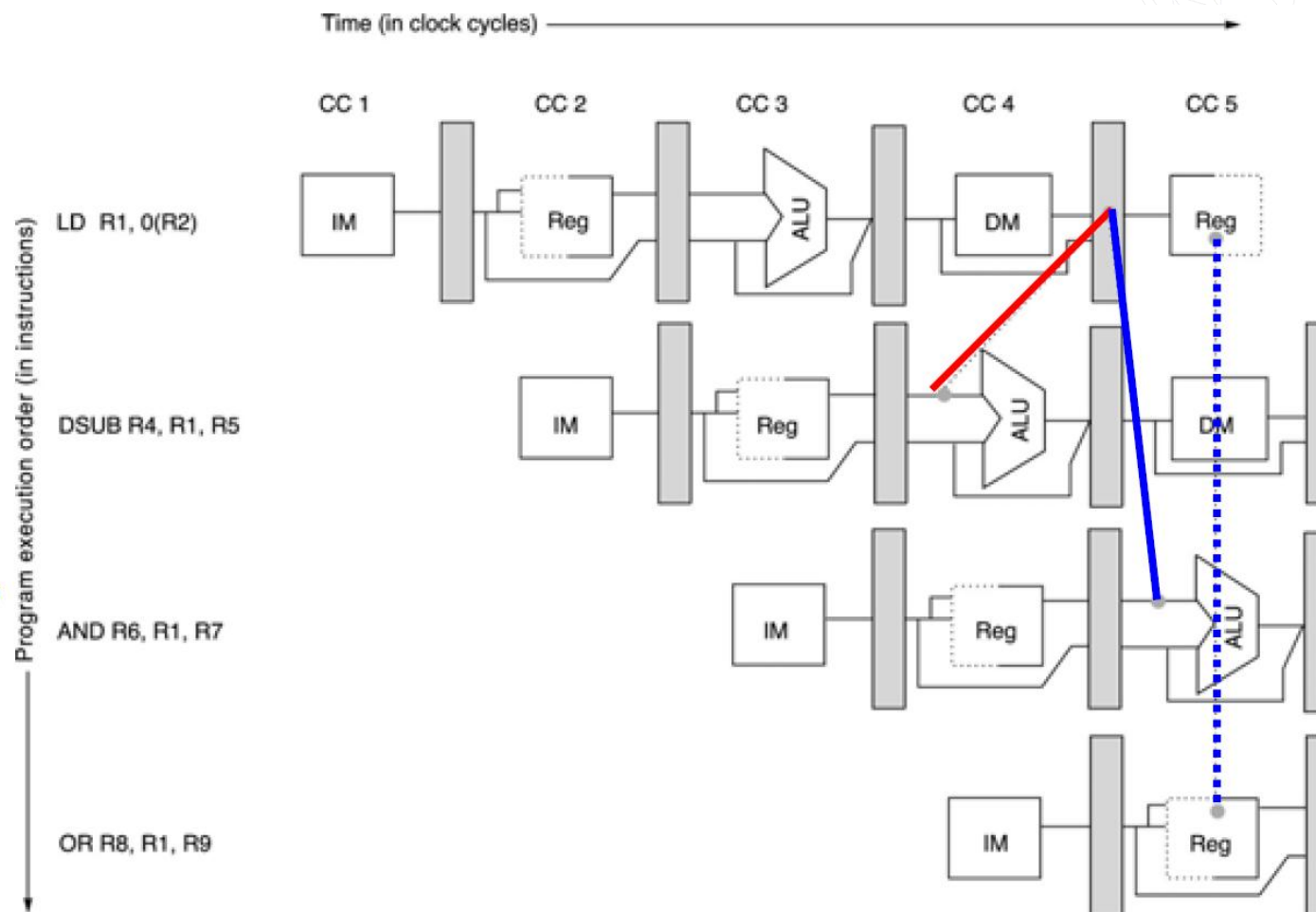
- The easiest solution is to stall the pipeline.
- We could delay the ~~AND~~ instruction by introducing a one-cycle delay into the pipeline, sometimes called a ~~bubble~~.



- Notice that we're still using forwarding in cycle 5, to get data from the MEM/WB pipeline register to the ALU.

处理数据冒险：流水线互锁

- ...when forwarding is not possible...
 - because we can't operate backward in time!
- ... we need a different HW solution
 - pipeline interlocks introduce stalls in the middle of an instruction

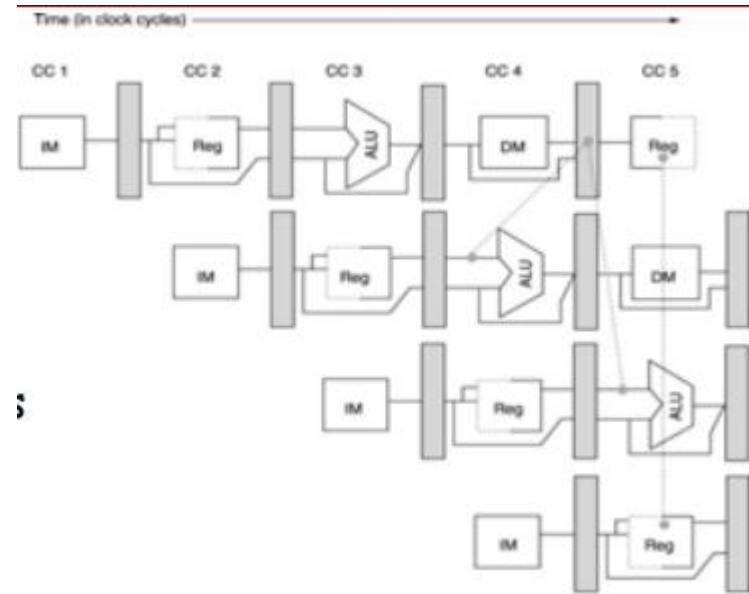


Interlocks are implemented using **NOPs**; a NOP is an instruction that does nothing. For instance **DADD R0, R0, R0**

由互锁导致的流水线停顿

At Cycle 4 the progression of **DSUB** (and all the following instructions) is halted

- no new instruction is fetched
- a NOP instruction is inserted into the EX stage and will exit the pipe at Cycle 6

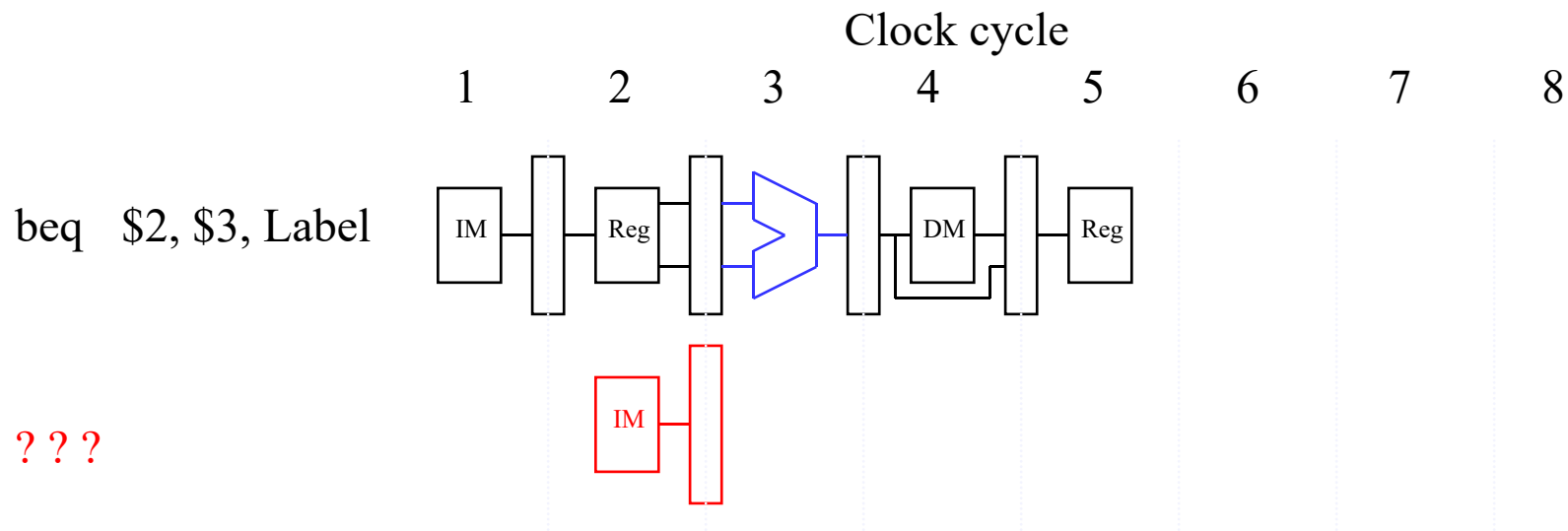


A **NOP Instruction** (a bubble) that is generated by one interlock activation increases the **CPI** of the stalled instruction by the length of **one clock cycle**

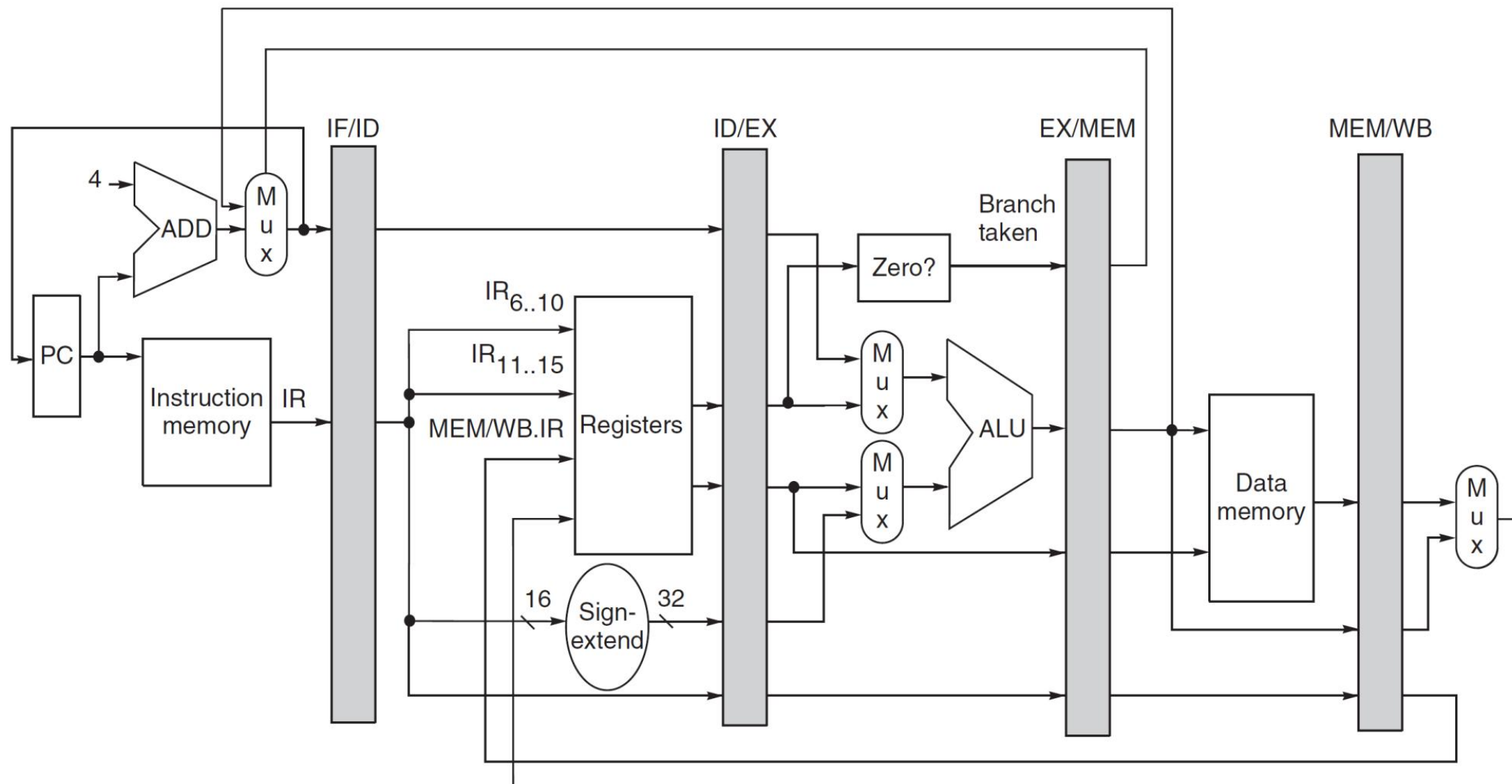
instruction	1	2	3	4	5	6	7	8	9
LD R1, 0(R2)	IF	ID	EX	MEM	WB				
DSUB R4, R1, R5		IF	ID	ID	EX	MEM	WB		
AND R6, R1, R7			IF	IF	ID	EX	MEM	WB	
STALL									
OR R8, R1, R9					IF	ID	EX	MEM	WB

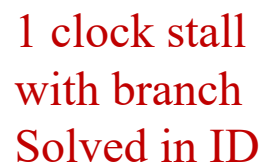
分支

- Most of the work for a **branch computation** is done in the **EX stage**.
 - The branch target address is computed.
 - The source registers are compared by the ALU, and the Zero flag is set or cleared accordingly.
- Thus, the **branch decision** cannot be made until the end of the **EX stage**.
 - But we need to know which instruction to fetch next, in order to keep the pipeline running!
 - This leads to what's called a **control hazard**.



MIPS 流水线实现





分支（控制）冒险及其对流水线性能的影响

Control hazards can cause a greater performance loss for a pipelined implementation than data hazards. In first approximation:

$$\text{speedupFromPipelining} = \frac{\text{pipelineDepth}}{1 + [\text{branchFrequency} \times \text{branchPenalty}]}$$

- performance loss can vary between 10% and 30% depending on the branch frequency
- generally, the deeper the pipeline, the worse the branch penalty (measured in clock cycles)
- Various strategies to reduce the **branch penalty**
 - static (compile time) schemes
 - they are fixed for each branch during the entire execution
 - dynamic
 - hardware and software techniques
 - more sophisticated branch prediction schemes

降低流水线分支开销（一）：冻结（冲刷）流水线

- Assuming now that the target address calculation and the comparison are performed during the ID stage, then, in case of “branch taken”, the PC is changed at the end of the ID stage
 - simplest strategy**: freezing the pipeline as soon as the instruction is detected as a branch and regardless of its outcome (**branch penalty is fixed, SW cannot reduce it**)

instruction	1	2	3	4	5	6	7	8	9
BNEZ R5, target	IF	ID							
fall-through inst		IF							
branch target			IF	ID	EX	MEM	WB		
branch target + 1				IF	ID	EX	MEM	WB	
branch target + 2					IF	ID	EX	MEM	WB

降低流水线分支代价（二）：预测「不跳转」

If the branch is taken, all the fetched instruction must be turned into NOPs and it is necessary to restart the fetch at the target address. The state must be unaffected!!

instruction	1	2	3	4	5	6	7	8	9
untaken branch	IF	ID							
Fall-Through Inst.		IF	ID	EX	MEM	WB			
FTI + 1			IF	ID	EX	MEM	WB		
FTI + 2				IF	ID	EX	MEM	WB	
FTI + 3					IF	ID	EX	MEM	WB

instruction	1	2	3	4	5	6	7	8	9
taken branch	IF	ID							
Fall-Through Inst.		IF							
branch target			IF	ID	EX	MEM	WB		
branch target + 1				IF	ID	EX	MEM	WB	
branch target + 2					IF	ID	EX	MEM	WB

降低流水线分支开销 (三) : 分支延迟

The instruction in the **delay slot** is executed whether or not the branch is taken

Scheduling problem for the compiler

Three strategies to fill the delay slot

- a) always best choice if possible
- b) preferred when branch is taken with high probability
 - e.g., loop branches
- c) dual case as (b)



Strategies (b) and (c) can be applied only if it is ok to execute the moved instruction if the prediction is wrong (i.e. the processor state is not affected)

总结

- Three kinds of hazards conspire to make pipelining difficult.
- **Structural hazards** result from not having enough hardware available to execute multiple instructions simultaneously.
 - These are avoided by adding more functional units (e.g., more adders or memories) or by redesigning the pipeline stages.
- **Data hazards** can occur when instructions need to access registers that haven't been updated yet.
 - Hazards from R-type instructions can be avoided with forwarding.
 - Loads can result in a “true” hazard, which must stall the pipeline.
- **Control hazards** arise when the CPU cannot determine which instruction to fetch next.
 - We can minimize delays by doing branch tests earlier in the pipeline.
 - We can also take a chance and predict the branch direction, to make the most of a bad situation.

示例

C.1 [15/15/15/15/25/10/15] <A.2> Use the following code fragment:

```
Loop:    LD      R1,0(R2)      ;load R1 from address 0+R2
         DADDI   R1,R1,#1      ;R1=R1+1
         SD      R1, 0(R2)     ;store R1 at address 0+R2
         DADDI   R2,R2,#4      ;R2=R2+4
         DSUB    R4,R3,R2      ;R4=R3-R2
         BNEZ    R4,Loop       ;branch to Loop if R4!=0
```

Assume that the initial value of R3 is $R2 + 396$.

- a. [15] <C.2> Data hazards are caused by data dependences in the code. Whether a dependency causes a hazard depends on the machine implementation (i.e., number of pipeline stages). List all of the data dependences in the code above. Record the register, source instruction, and destination instruction; for example, there is a data dependency for register R1 from the LD to the DADDI.

示例

a.

R1	LD	DADDI
R1	DADDI	SD
R2	LD	DADDI
R2	SD	DADDI
R2	DSUB	DADDI
R4	BNEZ	DSUB

示例

- b. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline without any forwarding or bypassing hardware but assuming that a register read and a write in the same clock cycle “forwards” through the register file, as shown in Figure C.6. Use a pipeline timing chart like that in Figure C.5. Assume that the branch is handled by flushing the pipeline. If all memory references take 1 cycle, how many cycles does this loop take to execute?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD R1, 0(R2)																		
DADDI R1, R1, #1																		
SD R1, 0(R2)																		
DADDI R2, R2, #4																		
DSUB R4, R3, R2																		
BNEZ R4, Loop																		
LD R1, 0(R2)																		

示例

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD	R1, 0(R2)	F	D	X	M	W													
DADDI	R1, R1, #1		F	s	s	D	X	M	W										
SD	R1, 0(R2)					F	s	s	D	X	M	W							
DADDI	R2, R2, #4								F	D	X	M	W						
DSUB	R4, R3, R2								F	s	s	D	X	M	W				
BNEZ	R4, Loop												F	s	s	D	X	M	W
LD	R1, 0(R2)																	F	D

Since the initial value of R3 is $R2 + 396$ and each instance of the loop adds 4 to R2, the total number of iterations is 99. Notice that there are 8 cycles lost to RAW hazards including the branch instruction. Two cycles are lost after the branch because of the instruction flushing. It takes 16 cycles between loop instances; the total number of cycles is $98 \times 16 + 18 = 1584$. The last loop takes two additional cycles since this latency cannot be overlapped with additional loop instances.

示例

- c. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline with full forwarding and bypassing hardware. Use a pipeline timing chart like that shown in Figure C.5. Assume that the branch is handled by predicting it as not taken. If all memory references take 1 cycle, how many cycles does this loop take to execute?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD R1, 0(R2)																		
DADDI R1, R1, #1																		
SD R1, 0(R2)																		
DADDI R2, R2, #4																		
DSUB R4, R3, R2																		
BNEZ R4, Loop																		
LD R1, 0(R2)																		

示例

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD	R1, 0(R2)	F	D	X	M	W													
DADDI	R1, R1, #1		F	D	s	X	M	W											
SD	R1, 0(R2)			F	s	D	X	M	W										
DADDI	R2, R2, #4					F	D	X	M	W									
DSUB	R4, R3, R2						F	D	X	M	W								
BNEZ	R4, Loop							F	s	D	X	M	W						
	(incorrect instruction)									F	s	s	s	s					
LD	R1, 0(R2)										F	D	X	M	W				

Again we have 99 iterations. There are two RAW stalls and a flush after the branch since the branch is taken. The total number of cycles is $9 \times 98 + 12 = 894$. The last loop takes three addition cycles since this latency cannot be overlapped with additional loop instances.

示例

- d. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline with full forwarding and bypassing hardware. Use a pipeline timing chart like that shown in Figure C.5. Assume that the branch is handled by predicting it as taken. If all memory references take 1 cycle, how many cycles does this loop take to execute?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD R1, 0(R2)																		
DADDI R1, R1, #1																		
SD R1, 0(R2)																		
DADDI R2, R2, #4																		
DSUB R4, R3, R2																		
BNEZ R4, Loop																		
LD R1, 0(R2)																		

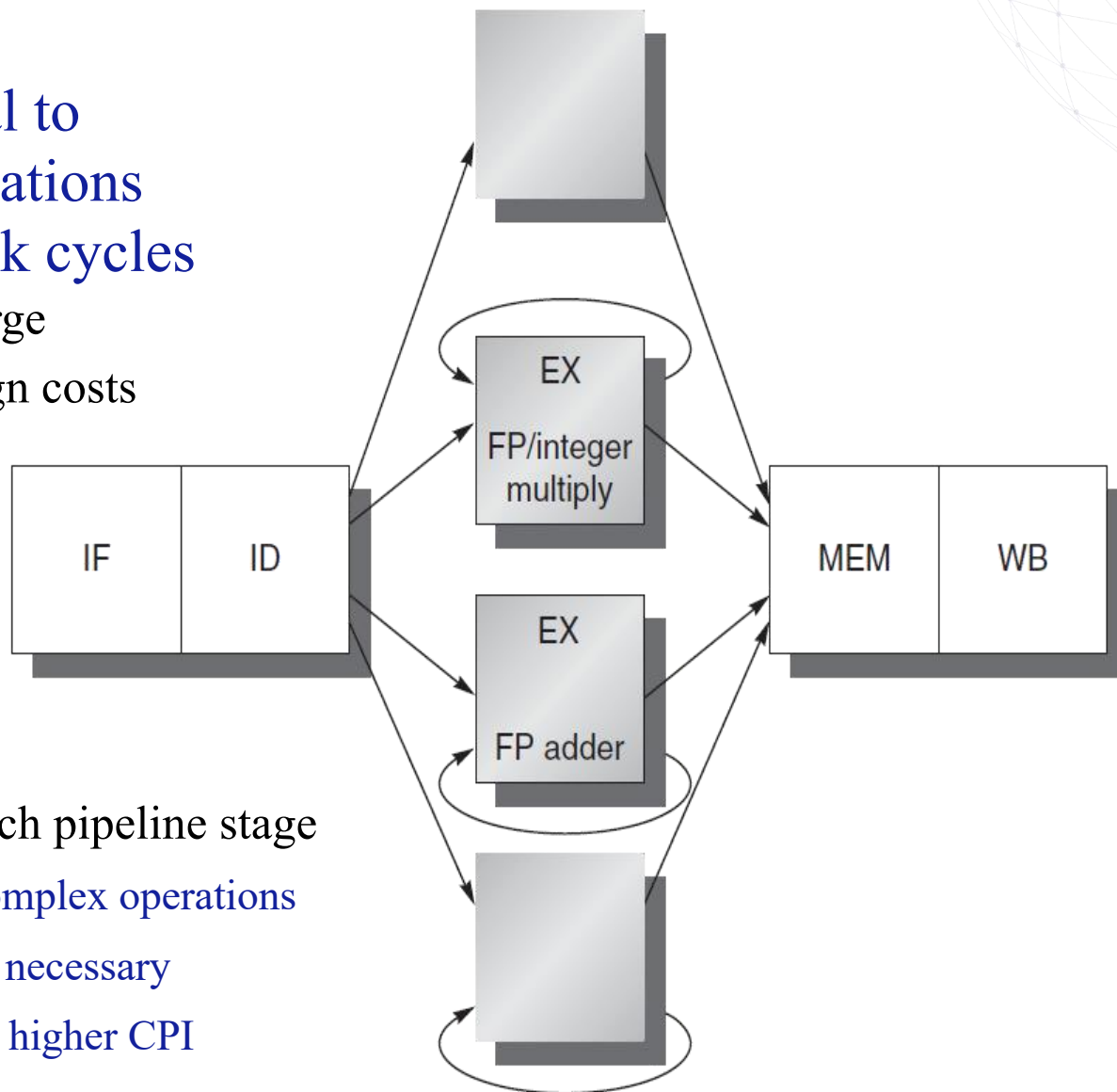
示例

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LD	R1, 0(R2)	F	D	X	M	W													
DADDI	R1, R1, #1		F	D	s	X	M	W											
SD	R1, 0(R2)			F	s	D	X	M	W										
DADDI	R2, R2, #4					F	D	X	M	W									
DSUB	R4, R3, R2						F	D	X	M	W								
BNEZ	R4, Loop							F	s	D	X	M	W						
LD	R1, 0(R2)									F	D	X	M	W					

Again we have 99 iterations. We still experience two RAW stalls, but since we correctly predict the branch, we do not need to flush after the branch. Thus, we have only $8 \times 98 + 12 = 796$.

流水线实现中多周期操作的处理

- Motivation: Impractical to require that all FP operations complete in 1 or 2 clock cycles
 - CCT would become too large
 - it would require huge design costs



- For higher clock rates
 - put fewer logic levels in each pipeline stage
 - more pipe stages for complex operations
 - more pipeline registers necessary
 - tradeoff lower CCT vs. higher CPI

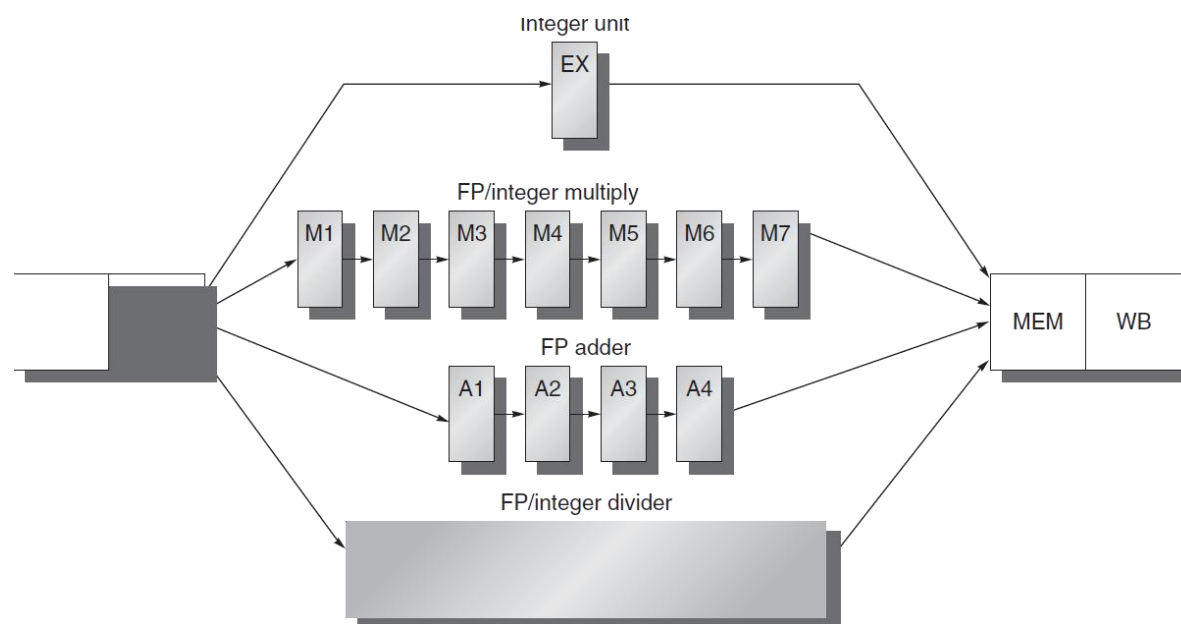
延迟与启动间隔

- **Two** important terms of pipelined operations
 - **Latency**: the number of intervening cycles between an instruction that produces a result and an instruction that uses the result.
 - **Initiation interval**: The number of cycles that must elapse between issuing two operations of a given type.

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

- Since most operations consume their operands at the beginning of EX, the latency is usually the # of stages after EX that an instruction produces a result
 - **Integer ALU operations** are all 0;
 - **Loads** are 1, since their results can be used after one intervening cycle.
 - **Stores**, which consume the value being stored 1 cycle later, are little different. The latency for the value being stored will be 1 cycle less.
- **Pipeline latency** is essentially equal to 1 cycle less than the depth of the execution pipeline, which is the number of stages from the EX stage to the stage that produces the result.

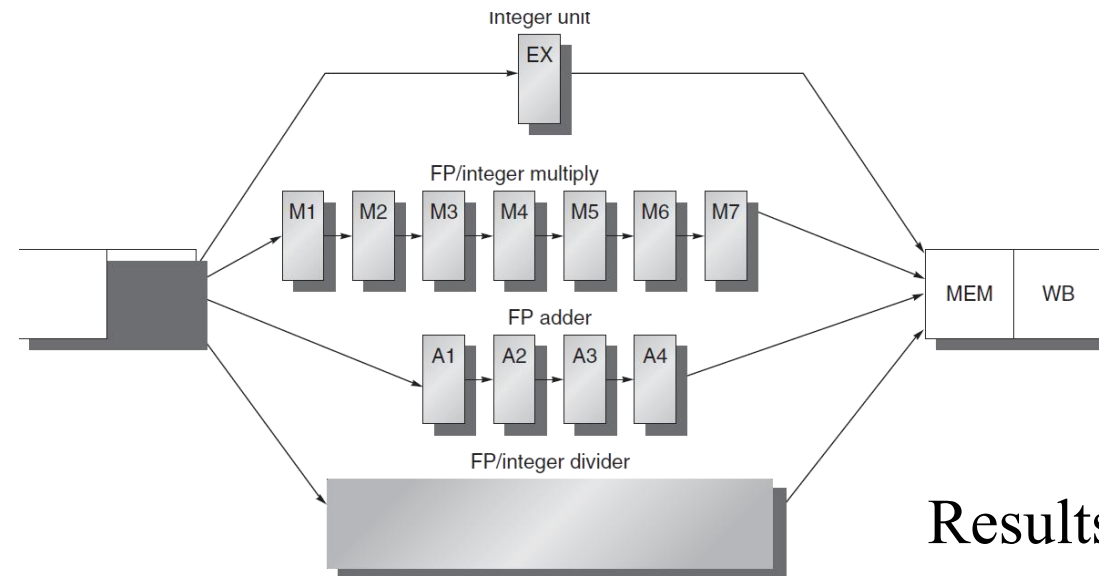
多级操作与多周期操作：即流水线级与顺序执行级



FunctionalUnit	(Additional)Latency	Initiation Interval
IntegerALU	0	1
Data Memory(Int&FP Loads)	1	1
FP Add	3	1
Int&FP Multiply	6	1
Int&FP Divide (SEQUENTIAL!)	24	25

一条支持多个未完成浮点操作的流水线

- The FP multiplier and adder are pipelined and have a depth of 7 and 4 stages.
- The FP divider is not pipelined, but requires 24 cycles to complete.
- For example, the 4th instruction after an FP add can use its result without being stalled. Similarly, 7th after an FP multiplier can use its result without being stalled.



MUL.D	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADD.D		IF	ID	A1	A2	A3	A4	MEM	WB		
L.D			IF	ID	EX	MEM	WB				
S.D				IF	ID	EX	MEM	WB			

Data are needed

Results are ready

由多个未完成操作引发的冒险

- Structural Hazards
 - sequential (non-pipelined) function units
 - e.g., two “close” DIV operations require stalling
 - number of register writes per cycle may be larger than 1
- More Frequent RAW Data Hazards
 - “instruction **y** (which follows instruction **x** in the program order) tries to read a source register before **x** writes it”
 - longer latency of operations produce more stalls
- Two New Classes of Data Hazards
 - WAW
 - “instruction **y** tries to write a destination register before **x** writes it”
 - WAR
 - “instruction **y** tries to write a destination register before **x** reads it”
- Out-of-order completion
 - makes exception handling more complex

由于多个未完成操作引起的冒险——写后写（WAW）冒险的示例

- WAW
 - “instruction **y** tries to write a destination register before **x** writes it”
- One may wonder how can WAW hazards occur since it may seem that a compiler would never generate two writes to the same register without an intervening read
- However such sequences do occur,
 - e.g., a program may take a path that the compiler cannot predict, e.g. due to an intervening branch
- Also, in the case of multiple-issue pipelines, an intervening read may be present (and stalled) while the second write could progress
- In this example if the branch is not taken a WAW if possible when the DIV completes its execution after the BEQZ, LD, and ADD complete theirs

```
DIV.D F1, F2, F4
BEQZ R2, target
LD F9, 0(R10)
ADD.D F1, F5, F6
...; other non-branch instructions
target: SUB.D F8, F9, F1
```

多个未完成操作引发的冒险 —— 结构冒险示例

Instruction	1	2	3	4	5	6	7	8	9	10	11
MUL.D F0, F4, F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADD R1, R3, R4		IF	ID	EX	MEM	WB					
ADD R2, R4, R5			IF	ID	EX	MEM	WB				
ADD.D F2, F4, F6				IF	ID	A1	A2	A3	A4	MEM	WB
ADD R6, R9, R7					IF	ID	EX	MEM	WB		
ADD R8, R3, R9						IF	ID	EX	MEM	WB	
L.D F2, 0(R2)							IF	ID	EX	MEM	WB

With Only 1 write port in register file, the processor must serialize the 3 WB instructions

- Option 1: Tracking port access during ID and (pre)-stalling corresponding instructions (same logic as for interlock detection)
- Option 2: Stall the conflicting instructions with smaller latency (“least-waited”) just before MEM or WB
 - Notice that there is no structural hazard at MEM because, among the 3 instructions, only L.D really needs to access memory. However, this assumes that there is separate HW to make the other two instructions progress into WB as they need

多个未完成操作引发的冒险——先读后写 (RAW) 冒险示例

Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L.D F4, 0(R2)	IF	ID	EX	MEM	WB												
MUL.D F0, F4, F6		IF	ID	●	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
ADD.D F2, F0, F8			IF	●	ID	●	●	●	●	●	●	A1	A2	A3	A4	MEM	WB
S.D F2, 0(R2)				●	IF	●	●	●	●	●	●	ID	EX	●	●	●	MEM

- Deeper pipelines increase the stall frequencies
 - Each instruction in the example depends on the previous one
 - Each instruction proceeds as soon as the awaited data become available (optimal forwarding)
 - To avoid a RAW hazard the S.D must wait for the ADD.D to provide the new value of F2 (via bypassing) before entering MEM
 - NB: Fig. C.37 is not consistent with C.38 since it implies that there is a structural hazard on MEM because ADD.D does not really need the MEM stage (but only to progress execution)

多个未完成操作引发的冒险——控制单元实现

- Assuming that all hazard detections are done in ID, before issuing an instruction **x** we must
 - check for structural hazards
 - the required function unit must be available (e.g., FP Divisor)
 - no future conflicts on the register write port
 - check for RAW data hazards
 - no conflict between **x** source register and previous instruction destination registers
 - e.g. if **x** is a FP operation on F2, check that F2 is not a destination in ID/A1, A1/A2, A2/A3
 - check for WAW data hazards
 - no instruction in A1,...,A4, D, M1,...M7 can have the same register destination as **x**
- Logic implementation is similar as for the MIPS integer pipeline

测试

Question 1:

(a) Circle one of A, B, C, D. As pipeline depth increases, the latency to process a single instruction; (b) Explain your reason

A. Decreases B. increases C. stays the same D. could increase, decrease, or stay the same, depending on

Question 2:

Spot all data dependencies (including ones that do not lead to stalls). Draw arrows from the stages where data is made available, directed to where it is needed.

(a) Circle the involved registers in the instructions. Assume no forwarding. One dependency has been drawn for you. How many stalls will we have to add to the pipeline to resolve the hazards?

(b) If allow forwarding, how many stalls will we have to add to the pipeline to resolve the hazards?

